

UNIVERSIDAD DE LOS ANDES

Facultad de Ingeniería y Ciencias Aplicadas

Arquitectura de Computadores

Proyecto 3

Implementación ASIC del Pochoco SoC

Del RTL al GDSII utilizando LibreLane y GF180MCU

Grupo 4

Integrantes: Christian Georgi
Diego Quinteros
Felipe Riveros

Profesor: Jorge Gómez Mir

Noviembre de 2026

Índice

1. Introducción **1**

1.1. Objetivos 1

1.2. Herramientas y parámetros de referencia 2

2. Del RTL de FPGA al RTL de ASIC **4**

2.1. Diseño original del Proyecto 2 4

2.2. Módulo superior `asic_top.v` 4

2.3. Problemas de compatibilidad encontrados 5

2.4. Justificación de los cambios 6

3. Memoria y programa del juego **8**

3.1. Programa `game.s` y generación automática de la ROM 8

3.2. Estrategia A: ROM combinacional y RAM de datos reducida 9

3.3. Estrategia B: Memoria unificada implementada con flip-flops 10

3.4. Comparación de las dos estrategias 10

3.5. Bootloader SPI 12

4. Síntesis y trazabilidad jerárquica **13**

4.1. Configuración de síntesis final 13

4.2. Las cinco celdas estándar más utilizadas 13

4.3. Área por módulo y comparación con la estimación inicial 14

4.4. Área del contador de ciclos 16

4.5. Método para identificar register file, ALU y contador 17

4.5.1. Identificación del banco de registros 18

4.5.2. Identificación de la ALU 19

4.5.3. Identificación del contador de ciclos 20

5. Análisis temporal (Timing) **23**

5.1. Configuración del análisis temporal 23

5.2. Barrido del período de reloj 24

5.3. Análisis de las esquinas temporales 25

5.4. Identificación de la ruta crítica 26
5.5. Optimización del diseño 27
5.6. Temporización del juego en Assembly 29
5.6.1. Espera inicial de tres segundos 30
5.6.2. Resolución temporal de 0,1 segundos 30

6. Floorplan, utilización y costos físicos 32
6.1. Configuración del floorplan 32
6.2. Área física y utilización del diseño 32
6.3. Distribución física de las celdas 34
6.4. Costo físico de la implementación de memoria 35
6.4.1. Memoria de datos de la arquitectura principal 36
6.4.2. Memoria de la arquitectura unificada 36
6.5. Comparación del costo físico entre arquitecturas 37

7. Verificación física y signoff 40
7.1. Objetivo de la verificación física 40
7.2. Verificación DRC 40
7.3. Verificación LVS 41
7.4. Verificación de antenas 42
7.5. Verificación temporal final 42
7.5.1. Verificación de setup 43
7.5.2. Verificación de hold 43
7.6. Resumen de los resultados de signoff 44
7.7. Conclusión de la verificación física 44

8. Simulación RTL y Gate-Level 46
8.1. Objetivo de la verificación funcional 46
8.2. Entorno y metodología de simulación 46
8.3. Simulación a nivel RTL 47
8.4. Simulación Gate-Level del ASIC 48
8.5. Análisis de las formas de onda 49

8.6. Comparación funcional entre RTL y Gate-Level 50
8.7. Alcance y limitaciones de la verificación 51
8.8. Conclusión de la verificación funcional 52

9. Conclusiones y trabajo futuro **53**
9.1. Conclusiones generales 53
9.2. Resultados de síntesis e implementación física 53
9.3. Cumplimiento temporal y frecuencia de operación 54
9.4. Evaluación de alternativas y optimizaciones 55
9.5. Verificaciones físicas y funcionales 56
9.6. Trabajo futuro 57
9.7. Conclusión final 57

10.Referencias bibliográficas **59**

1. Introducción

El diseño de circuitos integrados digitales permite implementar sistemas electrónicos especializados, considerando restricciones de área, rendimiento y temporización. A diferencia de una FPGA, donde la lógica se implementa mediante recursos programables, el diseño de un circuito integrado de aplicación específica (ASIC) requiere transformar una descripción de hardware en una estructura física compuesta por celdas estándar e interconexiones.

En el contexto del curso de Arquitectura de Computadores, el presente proyecto consiste en adaptar e implementar en tecnología ASIC el sistema `Pochoco SoC`, desarrollado previamente para una plataforma FPGA. Este sistema integra un procesador basado en la arquitectura RISC-V RV32E, memorias de instrucciones y datos, y periféricos de entrada y salida.

Como aplicación funcional, se utiliza un juego de reacción programado en lenguaje ensamblador, que interactúa con pulsadores, LEDs y visualizadores de siete segmentos. El procesador ejecuta las instrucciones del programa y controla los periféricos mediante operaciones de lectura y escritura en direcciones de memoria específicas.

La implementación se realiza mediante el flujo de diseño digital LibreLane, utilizando la tecnología GF180MCU. Este proceso comprende la adaptación del RTL, la síntesis lógica, la planificación física (*floorplan*), la colocación de celdas (*placement*), la distribución del reloj (*Clock Tree Synthesis*), el enrutamiento (*routing*) y las verificaciones físicas y temporales.

Adicionalmente, se analizan alternativas de organización de memoria y optimización RTL, con el propósito de estudiar su impacto sobre el área del circuito y el cumplimiento de las restricciones temporales. Finalmente, se realizan simulaciones RTL y *gate-level* para comprobar la consistencia funcional entre la descripción original y el netlist tecnológico generado.

1.1. Objetivos

El objetivo general del proyecto es implementar y verificar físicamente el sistema `Pochoco SoC` mediante un flujo de diseño ASIC, utilizando la tecnología GF180MCU y evaluando sus principales características de área, temporización y funcionamiento.

Para alcanzar este objetivo, se establecieron los siguientes objetivos específicos:

- Adaptar la arquitectura RTL del procesador RV32E y sus periféricos para permitir su implementación mediante celdas estándar.

- Integrar las memorias de instrucciones y datos necesarias para ejecutar el programa ensamblador del juego de reacción.
- Realizar la síntesis lógica y analizar los recursos utilizados, incluyendo celdas combinacionales, registros y estructuras de almacenamiento.
- Completar el flujo de implementación física mediante las etapas de *floorplan*, *placement*, distribución del reloj y *routing*.
- Evaluar el área ocupada, la utilización del núcleo y el cumplimiento de las restricciones temporales de setup y hold.
- Comparar alternativas de implementación y optimización para identificar sus efectos sobre el área y el rendimiento del circuito.
- Verificar la consistencia física del diseño mediante comprobaciones DRC y LVS.
- Realizar simulaciones RTL y *gate-level* para comparar el comportamiento observable del sistema antes y después de la síntesis tecnológica.

1.2. Herramientas y parámetros de referencia

La implementación del circuito se desarrolló utilizando herramientas de código abierto destinadas a la síntesis, implementación física, simulación y verificación de sistemas digitales.

Se seleccionó LibreLane como flujo principal de diseño ASIC y el PDK GF180MCU, variante `gf180mcuD`, junto con la biblioteca de celdas estándar `gf180mcu_fd_sc_mcu7t5v0`.

La configuración de referencia estableció un período objetivo de reloj de 100 ns, equivalente a una frecuencia de 10 MHz, y una utilización objetivo inicial del núcleo del 40 %.

La Tabla 1 resume las herramientas y parámetros utilizados.

Tabla 1: Herramientas y parámetros utilizados.

Elemento	**Versión o configuración**
LibreLane	v3.0.17
PDK	GF180MCU, variante `gf180mcuD`
Biblioteca estándar	`gf180mcu_fd_sc_mcu7t5v0`
Revisión del PDK	`792ef67037d74c8c51811eab3ce23cf793491296` (revisión identificada en la instalación utilizada)
Período objetivo	100 ns
Frecuencia objetivo	10 MHz
Utilización objetivo del core	40 %
Síntesis lógica	Yosys
Implementación física y STA	OpenROAD
Verificación DRC	Magic
Verificación LVS	Netgen
Simulación RTL y gate-level	Icarus Verilog
Visualización de formas de onda	GTKWave

Fuente: Elaboración propia.

El período objetivo de 100 ns y la utilización inicial del 40 % corresponden a los parámetros de configuración del flujo de implementación. Estos valores no deben confundirse con las métricas obtenidas al finalizar el proceso físico, particularmente la utilización final, que se analiza en los apartados posteriores.

Asimismo, la revisión del PDK presentada corresponde al identificador de la instalación utilizada durante las simulaciones tecnológicas. Su correspondencia exacta con la revisión empleada por LibreLane en la corrida física definitiva debe confirmarse mediante los registros de dicha ejecución.

2. Del RTL de FPGA al RTL de ASIC

2.1. Diseño original del Proyecto 2

El Proyecto 2 consistió en la implementación de un juego de reflejos sobre la FPGA Nandland Go Board, utilizando el procesador Espino Core integrado en el sistema Pochoco SoC. El diseño fue desarrollado en Verilog y permitió ejecutar un programa escrito en Assembly, encargado de controlar los LEDs, leer los botones y mostrar los resultados mediante dos displays de siete segmentos.

La arquitectura original utilizaba el módulo `game_top.v` como nivel superior, el cual instanciaba `pochoco_soc`. Este sistema integraba el procesador, una memoria RAM unificada para instrucciones y datos, los periféricos de entrada y salida, y una interfaz de comunicación SPI.

La memoria original se implementaba mediante el módulo `pochoco_ram.v`, utilizando un arreglo de registros de 32 bits con puertos independientes para instrucciones y datos. Su contenido se inicializaba con el archivo hexadecimal generado por el ensamblador, mediante la instrucción `$readmemh`.

El proceso de implementación estaba orientado a la FPGA Lattice iCE40 HX1K y utilizaba Yosys para la síntesis, nextpnr-ice40 para la ubicación y conexión de los recursos, e IceStorm para generar el archivo de programación.

Para el Proyecto 3 se conservó la arquitectura funcional del procesador y el programa del juego, pero fue necesario adaptar la organización del sistema y su implementación para utilizar un flujo de diseño ASIC con la tecnología GF180MCU.

2.2. Módulo superior `asic_top.v`

Para la implementación ASIC se creó el módulo `asic_top.v`, que reemplaza a `game_top.v` como módulo superior del diseño físico.

Este módulo permite integrar el Pochoco SoC con una interfaz de entrada y salida independiente de los recursos específicos de la FPGA. Conserva las conexiones correspondientes a los cuatro botones, cuatro LEDs, dos displays de siete segmentos y la interfaz SPI.

Una de las principales modificaciones fue la incorporación de una entrada externa de reset activo en bajo, denominada `i_Rst_n`. En la arquitectura ASIC, esta señal se conecta directamente al sistema mediante `rst_ni`, eliminando la dependencia del mecanismo de inicialización utilizado anteriormente en la FPGA.

Adicionalmente, se incorporó el parámetro `SIMULATION`, que permite seleccionar la

memoria de instrucciones correspondiente al funcionamiento normal o a las pruebas de simulación.

El módulo `asic_top.v` instancia un único `pochoco_soc`, manteniendo las conexiones funcionales del procesador y los periféricos. De esta manera, las modificaciones necesarias para la tecnología ASIC se concentran principalmente en el nivel superior, el sistema de reset y la organización de las memorias.

La implementación física se configuró mediante LibreLane, utilizando el PDK `gf180mcuD` y la biblioteca de celdas estándar correspondiente. El módulo `asic_top` se estableció como diseño principal para la síntesis y el flujo de implementación física.

2.3. Problemas de compatibilidad encontrados

La adaptación del diseño desde FPGA a ASIC requirió revisar las construcciones del RTL relacionadas con la inicialización, las memorias y la integración de los módulos.

Uno de los principales aspectos fue el uso de una memoria unificada inicializada mediante `$readmemh`. Esta implementación resultaba adecuada para el flujo FPGA utilizado anteriormente, pero requería una alternativa que permitiera sintetizar el contenido del programa mediante celdas estándar en el flujo ASIC seleccionado.

Para resolverlo, se implementó una arquitectura con una ROM de instrucciones y una RAM de datos independientes. La ROM se construyó mediante una descripción combinacional basada en `case`, mientras que la lectura de instrucciones se mantuvo sincronizada con el reloj para conservar el comportamiento temporal esperado por el procesador.

La RAM de datos se implementó mediante registros sintetizables, utilizando una profundidad de 64 palabras de 32 bits y conservando la escritura selectiva por bytes.

También fue necesario modificar el sistema de reset para utilizar una entrada externa y definir un nuevo módulo superior que pudiera ser utilizado directamente por LibreLane.

La Tabla 2 resume las principales construcciones revisadas y las modificaciones implementadas.

Tabla 2: Construcciones del RTL original y adaptación a ASIC.

Construcción	**Problema**	**Cambio implementado**	**Log y línea**
Inicialización con `$readmemh`	Dependencia de un archivo hexadecimal para inicializar la memoria unificada del diseño FPGA.	Implementación de una ROM de instrucciones descrita mediante `case`, sintetizable como lógica combinacional.	`rtl/game_rom.v`, líneas 3–98.
Memoria RAM unificada	La arquitectura original compartía el almacenamiento de instrucciones y datos, por lo que se evaluó su adaptación al flujo de celdas estándar.	Separación en una ROM de instrucciones y una RAM de datos de 64 palabras de 32 bits.	`rtl/pochoco_soc.v`, líneas 106–127.
Reset de la FPGA	La inicialización del sistema original dependía del mecanismo de reset incorporado al SoC para su funcionamiento en FPGA.	Incorporación de una entrada externa `i_Rst_n` para controlar el reset del diseño ASIC.	`rtl/asic_top.v`, líneas 3–5 y 31–35.
Módulo superior `game_top`	El módulo original estaba orientado a la implementación y programación de la Nandland Go Board.	Creación de `asic_top.v` como módulo superior del flujo ASIC.	`rtl/asic_top.v`, líneas 1–63.
Lectura de instrucciones	La separación de memorias requería conservar la latencia de lectura esperada por el procesador.	Incorporación de una salida registrada, actualizada en el flanco positivo del reloj.	`rtl/pochoco_instruccion_rom.v`, líneas 30–34.
Selección de programa	Las pruebas del ASIC requerían ejecutar programas específicos sin modificar manualmente la arquitectura principal.	Utilización del parámetro `SIMULATION` para seleccionar entre `game_rom` y `game_sim_rom`.	`rtl/pochoco_instruccion_rom.v`, líneas 11–28.

Fuente: Elaboración propia.

2.4. Justificación de los cambios

Las modificaciones realizadas tuvieron como objetivo permitir que la arquitectura desarrollada para FPGA pudiera sintetizarse e implementarse físicamente utilizando celdas estándar de la tecnología GF180MCU, conservando el comportamiento funcional del procesador y del juego.

La incorporación de `asic_top.v` permitió definir una interfaz de hardware independiente de la placa FPGA y establecer un módulo superior compatible con el flujo de síntesis y diseño físico de LibreLane. Asimismo, la utilización de un reset externo facilitó el control de la inicialización durante las simulaciones y las verificaciones del sistema.

La separación de las memorias permitió implementar el programa del juego mediante lógica ROM y mantener una RAM de menor capacidad para las operaciones de lectura y escritura de datos. Esta organización evita utilizar una memoria de instrucciones completamente escribible cuando el programa permanece fijo durante la ejecución.

La ROM generada mediante `case` permite que las instrucciones sean interpretadas por las herramientas de síntesis como una descripción de lógica combinacional. Por otra parte, mantener la salida registrada conserva la latencia de lectura prevista por el procesador, reduciendo la necesidad de modificar su arquitectura interna.

Finalmente, la parametrización de la memoria de instrucciones permitió utilizar ver-

siones específicas del programa durante las simulaciones. Esto facilitó la validación del diseño antes y después de la síntesis.

En conjunto, estos cambios permitieron conservar la funcionalidad principal del Proyecto 2 y preparar el sistema para su implementación, análisis temporal y verificación física como ASIC.

3. Memoria y programa del juego

3.1. Programa `game.s` y generación automática de la ROM

El programa del juego de reflejos fue desarrollado en lenguaje Assembly, utilizando instrucciones compatibles con el procesador Espino Core. Su funcionamiento incluye la inicialización del sistema, el control de los LEDs y botones, la selección pseudoaleatoria del LED objetivo, la medición del tiempo de reacción y el cálculo del promedio al completar diez respuestas correctas.

El código fuente se encuentra almacenado en el archivo `sw/game.s`. Para convertirlo a instrucciones ejecutables por el procesador se utiliza el ensamblador `sw/assembler.py`, desarrollado durante el Proyecto 2.

Este ensamblador procesa las instrucciones y etiquetas del programa, calcula las direcciones correspondientes y genera el archivo `game.hex`, que contiene una palabra hexadecimal de 32 bits por cada instrucción.

Para comprobar el tamaño del programa se contabilizaron las palabras almacenadas en `game.hex`, obteniéndose un total de 86 instrucciones, equivalentes a 344 bytes de código máquina.

Para la implementación ASIC, el programa se incorporó a una memoria ROM descrita mediante lógica combinacional. Esta solución permite representar las instrucciones como constantes dentro del diseño, evitando implementar una memoria de instrucciones completamente escribible mediante flip-flops.

El módulo `game_rom.v` contiene las instrucciones organizadas mediante una estructura `case`, que relaciona cada dirección con la palabra de 32 bits correspondiente.

La ROM se integra mediante `pochoco_instruccion_rom.v`, encargado de entregar las instrucciones al procesador conservando la temporización requerida por la etapa de búsqueda.

La Tabla 3 presenta el tamaño efectivo de los recursos utilizados por el programa.

Tabla 3: Tamaño efectivo de los recursos del juego.

Recurso	**Cantidad de palabras**
Instrucciones del juego	86
Datos utilizados en RAM	0

Fuente: Elaboración propia.

La cantidad de instrucciones se determinó a partir del archivo hexadecimal generado por el ensamblador. Para determinar el número de palabras de datos efectivamente

utilizadas, es necesario identificar las direcciones de memoria RAM accedidas por el programa, diferenciándolas de los registros internos del procesador y de los periféricos mapeados en memoria.

3.2. Estrategia A: ROM combinacional y RAM de datos reducida

La primera alternativa implementada consiste en separar la memoria de instrucciones de la memoria de datos, utilizando una ROM combinacional y una RAM de capacidad reducida.

La memoria de instrucciones contiene el programa del juego y se implementa mediante el módulo `game_rom.v`. Debido a que las instrucciones permanecen constantes durante la ejecución, no se requiere implementar capacidad de escritura sobre esta memoria.

Por otra parte, la memoria de datos se implementa mediante `pochoco_data_ram.v`, configurada con una capacidad de 64 palabras de 32 bits, equivalentes a 2048 bits o 256 bytes de almacenamiento.

Esta RAM permite realizar operaciones de lectura y escritura, incluyendo escrituras selectivas por byte, según las señales de control entregadas por la unidad de carga y almacenamiento del procesador.

Ambas memorias se integran dentro de `pochoco_soc.v`. La memoria ROM proporciona las instrucciones que ejecuta el procesador, mientras que la RAM se utiliza para almacenar y recuperar datos durante el funcionamiento.

La separación de memorias permite reducir la cantidad de almacenamiento implementado mediante flip-flops, ya que las instrucciones del programa se representan mediante lógica combinacional en lugar de utilizar posiciones de memoria escribibles.

Para la implementación ASIC se utilizó el módulo superior `asic_top.v`, junto con la configuración `librelane/config.json`.

La síntesis y el diseño físico se realizaron utilizando LibreLane y la tecnología GF180MCU, con un período de reloj objetivo de 100 ns y una utilización inicial de floorplan del 40 %.

Esta arquitectura fue seleccionada como alternativa principal para la implementación física final del ASIC.

3.3. Estrategia B: Memoria unificada implementada con flip-flops

La segunda alternativa consiste en utilizar una memoria unificada para almacenar tanto las instrucciones del programa como los datos necesarios durante su ejecución.

Esta arquitectura se implementó mediante `pochoco_unified_ram.v`, integrado dentro del módulo `pochoco_soc_unified.v`.

A diferencia de la estrategia anterior, la memoria de instrucciones y la memoria de datos comparten un mismo espacio de almacenamiento, implementado mediante flip-flops y lógica de control y selección.

Para los experimentos ASIC se configuró una memoria unificada de 128 palabras de 32 bits, equivalentes a 4096 bits o 512 bytes de almacenamiento.

Esta organización permite disponer de una memoria escribible para instrucciones y datos, manteniendo los mecanismos de lectura y escritura utilizados por el procesador.

Sin embargo, al implementar el almacenamiento mediante celdas estándar, cada palabra requiere elementos secuenciales adicionales, junto con la lógica necesaria para seleccionar direcciones, controlar las escrituras y entregar los datos solicitados.

La implementación se configuró mediante el archivo `librelane/config_unified.json`, utilizando el módulo superior `pochoco_soc_unified` y la misma tecnología GF180MCU empleada en la estrategia A.

También se realizaron experimentos con diferentes períodos de reloj para evaluar el comportamiento temporal de esta organización.

El objetivo de desarrollar esta segunda alternativa fue comparar una memoria completamente implementada mediante flip-flops con una arquitectura que utiliza lógica ROM para las instrucciones y mantiene una RAM de datos reducida.

3.4. Comparación de las dos estrategias

Las dos arquitecturas fueron sintetizadas e implementadas físicamente utilizando LibreLane y la tecnología GF180MCU.

Para realizar una comparación bajo condiciones equivalentes, se consideraron las corridas de ambas arquitecturas con un período de reloj objetivo de 100 ns y una utilización inicial de floorplan del 40 %.

Las métricas evaluadas corresponden al área de celdas estándar, la cantidad de celdas secuenciales, la utilización física final y el comportamiento temporal obtenido mediante el análisis de timing.

Los resultados se presentan en la Tabla 4.

Tabla 4: Comparación de alternativas de memoria.

Métrica	**ROM + RAM**	**Unificada**
Área de celdas (μm^2)	461 844	877 588
Número de flip-flops	2851	4914
Utilización (%)	51,17	52,01
Frecuencia máxima verificada (MHz)	14,29	No determinada

Fuente: Elaboración propia.

El número de flip-flops se verificó mediante el análisis de los netlists sintetizados por Yosys. Para la arquitectura ROM + RAM se identificaron 2654 flip-flops tipo D, 194 con reset y 3 con set, totalizando 2851 elementos. En la arquitectura unificada se identificaron 4708 flip-flops tipo D, 203 con reset y 3 con set, alcanzando un total de 4914 elementos. En los netlists analizados no se identificaron latches, y ambos resultados coinciden con las métricas de celdas secuenciales entregadas por LibreLane.

Los resultados muestran que la arquitectura ROM + RAM presentó un área de celdas estándar de 461 844 μm^2, mientras que la memoria unificada alcanzó 877 588 μm^2.

La diferencia corresponde a 415 744 μm^2, lo que representa un incremento aproximado del 90 % en el área de celdas de la arquitectura unificada respecto de la alternativa ROM + RAM.

En cuanto a los elementos secuenciales, la estrategia A utilizó 2851 celdas y la estrategia B alcanzó 4914, presentando un aumento de 2063 celdas, equivalente aproximadamente al 72,4 %.

Esta diferencia se relaciona principalmente con la implementación de una memoria unificada mediante flip-flops, en lugar de almacenar las instrucciones mediante lógica combinacional.

La utilización física final fue de 51,17 % para la arquitectura ROM + RAM y de 52,01 % para la memoria unificada.

Aunque ambas implementaciones presentan porcentajes de utilización similares, esto no significa que ocupen la misma superficie. La arquitectura unificada requiere un núcleo físico de mayor tamaño debido al incremento de celdas necesarias para implementar el almacenamiento y su lógica asociada.

Respecto al análisis temporal, la arquitectura ROM + RAM obtuvo un Worst Negative Slack (WNS) de +28,588 ns con un período de reloj de 100 ns, cumpliendo las restricciones temporales de setup.

Adicionalmente, se verificó el cumplimiento temporal con un período de 70 ns, equivalente a una frecuencia de aproximadamente 14,29 MHz. Este valor corresponde a la mayor frecuencia verificada que cumple timing dentro de los experimentos realizados, sin representar necesariamente el límite máximo absoluto de la arquitectura.

Por otra parte, la memoria unificada obtuvo un WNS de -0,319 ns para un período de 100 ns, indicando una pequeña violación temporal de setup.

Las corridas realizadas con períodos de 70 ns y 50 ns también presentaron violaciones temporales, con valores de WNS de -29,230 ns y -24,747 ns, respectivamente.

Por lo tanto, no se determinó experimentalmente una frecuencia máxima verificada que cumpliera todas las restricciones temporales para la arquitectura unificada.

En consecuencia, bajo las condiciones evaluadas, la arquitectura ROM + RAM presentó ventajas en términos de área de implementación, cantidad de celdas secuenciales y cumplimiento de las restricciones temporales.

Por estas razones, se seleccionó esta alternativa como arquitectura principal para la implementación física final del ASIC.

3.5. Bootloader SPI

Como parte de la arquitectura experimental con memoria unificada se incorporó el módulo `pochoco_boot_loader.v`, destinado a permitir la carga de instrucciones a través de una interfaz SPI.

Este módulo se relaciona con `pochoco_spi_slave.v`, encargado de recibir la información transmitida mediante las señales de comunicación SPI.

El propósito del bootloader es permitir la incorporación de instrucciones a la memoria unificada sin necesidad de modificar el contenido de una ROM combinacional y volver a sintetizar todo el diseño.

Esta alternativa proporciona mayor flexibilidad para modificar el programa almacenado, pero también incorpora lógica adicional de control y requiere una memoria capaz de almacenar información escribible.

El bootloader se incluyó como parte de la arquitectura experimental de memoria unificada. En cambio, la implementación principal del ASIC utiliza una ROM que contiene el programa incorporado mediante lógica combinacional.

Por lo tanto, la carga de instrucciones mediante SPI constituye una funcionalidad complementaria de la estrategia B y no forma parte del mecanismo de almacenamiento utilizado en la arquitectura ROM + RAM seleccionada para el diseño físico final.

4. Síntesis y trazabilidad jerárquica

4.1. Configuración de síntesis final

La síntesis del diseño ASIC se realizó mediante Yosys, integrado en el flujo de LibreLane, utilizando la tecnología GF180MCU y el PDK `gf180mcuD`. La biblioteca de celdas estándar seleccionada fue `gf180mcu_fd_sc_mcu7t5v0`.

El módulo superior utilizado fue `asic_top.v`, que integra el procesador Espino Core, las memorias de instrucciones y datos, los periféricos y la interfaz SPI. Para la implementación principal se utilizó la arquitectura con ROM combinacional y RAM de datos reducida.

La configuración de síntesis se estableció mediante el archivo `librelane/config.json`, considerando un período de reloj de 100 ns y una utilización inicial del núcleo del 40 %.

Durante la síntesis, Yosys convierte la descripción RTL en un netlist tecnológico compuesto por celdas estándar de la biblioteca seleccionada. Este netlist contiene los elementos secuenciales y combinacionales necesarios para implementar las funciones descritas en Verilog.

La implementación física definitiva se obtuvo mediante la corrida `fisico_rom_100ns_actualizado`. El diseño final presentó un área de celdas estándar de 461 844 μm^2, una utilización física de 51,17 % y un total de 2851 flip-flops.

Adicionalmente, se realizó una síntesis con jerarquía preservada utilizando `librelane/config_hierar` Esta configuración permitió identificar las celdas correspondientes a los módulos individuales del procesador y obtener estimaciones de área por componente.

4.2. Las cinco celdas estándar más utilizadas

Para identificar los recursos tecnológicos predominantes se analizó el netlist generado por Yosys durante la síntesis de la arquitectura ROM + RAM.

El procedimiento consistió en contabilizar las instancias de cada tipo de celda estándar presentes en el módulo superior `asic_top`.

En total se identificaron 11 324 instancias tecnológicas en el netlist de síntesis analizado. Las cinco celdas más frecuentes se presentan en la Tabla 5.

Tabla 5: Cinco tipos de celdas más frecuentes en la síntesis final.

Celda	**Cantidad**	**Explicación microarquitectónica**
`dffq_1`	2654	Flip-flop tipo D utilizado para almacenar información en registros internos y memorias del sistema.
`mux2_2`	2240	Multiplexor de dos entradas utilizado para seleccionar datos, direcciones y señales dentro del procesador.
`nand2_1`	1689	Compuerta NAND de dos entradas utilizada en lógica combinacional, decodificación y control.
`oai21_1`	700	Compuerta OR-AND-Invert utilizada para implementar funciones lógicas y combinar condiciones de selección y control.
`nand3_1`	693	Compuerta NAND de tres entradas utilizada para combinar señales lógicas y condiciones de control.

Fuente: Elaboración propia.

La celda más frecuente fue `dffq_1`, con 2654 instancias. Su elevada presencia se relaciona principalmente con los elementos de almacenamiento del procesador y de la RAM de datos.

En segundo lugar, se identificaron 2240 multiplexores `mux2_2`, utilizados para seleccionar información entre diferentes fuentes. Su presencia se relaciona con la selección de datos en memorias, registros y unidades de procesamiento.

Las compuertas `nand2_1`, `oai21_1` y `nand3_1` implementan distintas funciones combinacionales necesarias para las operaciones del procesador.

Los resultados muestran que una parte importante de la implementación está asociada al almacenamiento de información y a la lógica de selección de datos.

4.3. Área por módulo y comparación con la estimación inicial

Para analizar la distribución del área se utilizó una síntesis que conserva la jerarquía de los módulos del diseño.

Este procedimiento permite identificar la contribución de componentes como el banco de registros, la ALU, las etapas del procesador, las memorias y los periféricos.

La Tabla 6 presenta las áreas obtenidas mediante el reporte `stat.json` generado por Yosys durante la síntesis jerárquica.

Tabla 6: Distribución del área por módulo según síntesis jerárquica.

Módulo	**Área (μm^2)**
Pochoco SoC completo	367 443,55
Espino Core completo	107 946,76
Banco de registros	73 765,31
Etapa de decodificación	22 388,84
ALU	10 152,80
Etapa de búsqueda de instrucciones	8 510,79
Unidad de carga y almacenamiento	3 211,58
Decodificador	2 214,96
Unidad de control	70,25
RAM de datos	232 746,08
ROM de instrucciones	8 205,66
Periféricos	9 709,37
Interfaz SPI	6 276,08

Fuente: Elaboración propia.

La síntesis jerárquica presentó un área total estimada de 367 443,55 μm^2 para el módulo superior.

Entre los principales componentes destaca la RAM de datos, con 232 746,08 μm^2, seguida por el procesador Espino Core, con 107 946,76 μm^2.

Dentro del procesador, el banco de registros presentó un área de 73 765,31 μm^2, mientras que la etapa de decodificación y la ALU alcanzaron 22 388,84 μm^2 y 10 152,80 μm^2, respectivamente.

Es importante considerar que las áreas de los módulos superiores incluyen las de sus submódulos. Por lo tanto, los valores presentados no deben sumarse directamente, debido a que esto produciría una doble contabilización.

Para comparar la estimación jerárquica con el resultado físico final, se consideraron las siguientes áreas:

$$A_{\text{jerarquica}} = 367\,443,55\ \mu\text{m}^2 \tag{1}$$

$$A_{\text{final}} = 461\,844\ \mu\text{m}^2 \tag{2}$$

La diferencia de área se calculó mediante:

$$\Delta A = A_{\text{final}} - A_{\text{jerarquica}} \tag{3}$$

$$\Delta A = 461\,844 - 367\,443,55 = 94\,400,45\ \mu\text{m}^2 \tag{4}$$

El incremento relativo corresponde a:

$$\Delta A_{\,\%} = \frac{A_{\text{final}} - A_{\text{jerarquica}}}{A_{\text{jerarquica}}} \cdot 100 \tag{5}$$

$$\Delta A_{\,\%} \approx 25,69\,\% \tag{6}$$

La implementación física final presentó un área de celdas aproximadamente un 25,69 % superior a la estimación jerárquica.

Esta diferencia se relaciona con las condiciones de optimización de ambas síntesis y con las modificaciones efectuadas durante el flujo físico, incluyendo la incorporación de buffers, la distribución del reloj y las reparaciones temporales.

Por lo tanto, la síntesis jerárquica permite estudiar la contribución de cada módulo, mientras que las métricas físicas finales representan el resultado obtenido después de la implementación y optimización del circuito.

4.4. Área del contador de ciclos

El contador de ciclos se encuentra implementado dentro del módulo `pochoco_periph.v`. Su función consiste en registrar la cantidad de ciclos de reloj transcurridos desde la liberación del reset.

El valor del contador aumenta en una unidad por cada ciclo y puede ser consultado por el procesador mediante la dirección de memoria mapeada `0x8000000C`.

Su implementación se compone de un registro de 32 bits y lógica combinacional encargada de calcular el siguiente valor.

Para determinar el área asociada al contador se analizaron los elementos de almacenamiento en OpenROAD y la lógica combinacional de actualización en el netlist de síntesis jerárquica.

Mediante la base de datos física final se identificaron 32 flip-flops correspondientes a las instancias `_19748_` hasta `_19779_`.

Todas estas instancias utilizan la celda estándar `gf180mcu_fd_sc_mcu7t5v0__dffrnq_1`, con un área individual de 74,6368 μm^2.

El área de almacenamiento del contador se calculó mediante:

$$A_{\text{FF}} = N_{\text{FF}} \cdot A_{\text{celda}} \tag{7}$$

Reemplazando:

$$A_{\text{FF}} = 32 \cdot 74,6368 = 2388,38\ \mu\text{m}^2 \tag{8}$$

Posteriormente, se analizaron las conexiones del netlist jerárquico que alimentan las entradas de los flip-flops del contador.

Este procedimiento permitió identificar 78 celdas combinacionales, entre ellas compuertas XOR, XNOR, NAND, NOR, AND y AOI.

El área de estas celdas se calculó utilizando sus dimensiones físicas, obtenidas desde la biblioteca LEF de GF180MCU.

La suma de las áreas combinacionales identificadas fue:

$$A_{\text{comb}} = 1505,91\ \mu\text{m}^2 \tag{9}$$

Por lo tanto, el área conjunta de los flip-flops y la lógica combinacional identificada es:

$$A_{\text{identificada}} = A_{\text{FF}} + A_{\text{comb}} \tag{10}$$

$$A_{\text{identificada}} = 2388,3776 + 1505,9072 = \boxed{3894,28\ \mu\text{m}^2} \tag{11}$$

De este valor, aproximadamente el 61,3 % corresponde a los elementos de almacenamiento y el 38,7 % a la lógica combinacional identificada.

Se debe considerar que el área de los flip-flops se obtuvo del layout físico final, mientras que la lógica de actualización se identificó mediante una síntesis jerárquica.

Por lo tanto, el valor de 3894,28 μm^2 representa una estimación basada en las celdas identificadas y no una medición exclusiva del contador dentro del layout final. Tampoco se atribuye íntegramente al contador la lógica de lectura compartida con los demás periféricos.

4.5. Método para identificar register file, ALU y contador

Para relacionar los componentes del código RTL con sus correspondientes celdas tecnológicas se utilizaron los netlists generados por Yosys y las bases de datos físicas de OpenROAD.

Este procedimiento permitió localizar los elementos asociados al banco de registros, la unidad aritmético-lógica y el contador de ciclos.

4.5.1. Identificación del banco de registros

El banco de registros forma parte del procesador Espino Core y permite almacenar los operandos y resultados utilizados durante la ejecución del programa.

Mediante el análisis del netlist físico final se identificaron 480 flip-flops correspondientes al almacenamiento de los registros del procesador.

Para visualizar estas instancias se utilizó el script `scripts/resaltar_banco.tcl`, que carga la base de datos física del ASIC y resalta los elementos correspondientes.

La verificación realizada en OpenROAD permitió identificar las 480 instancias esperadas dentro del layout definitivo.

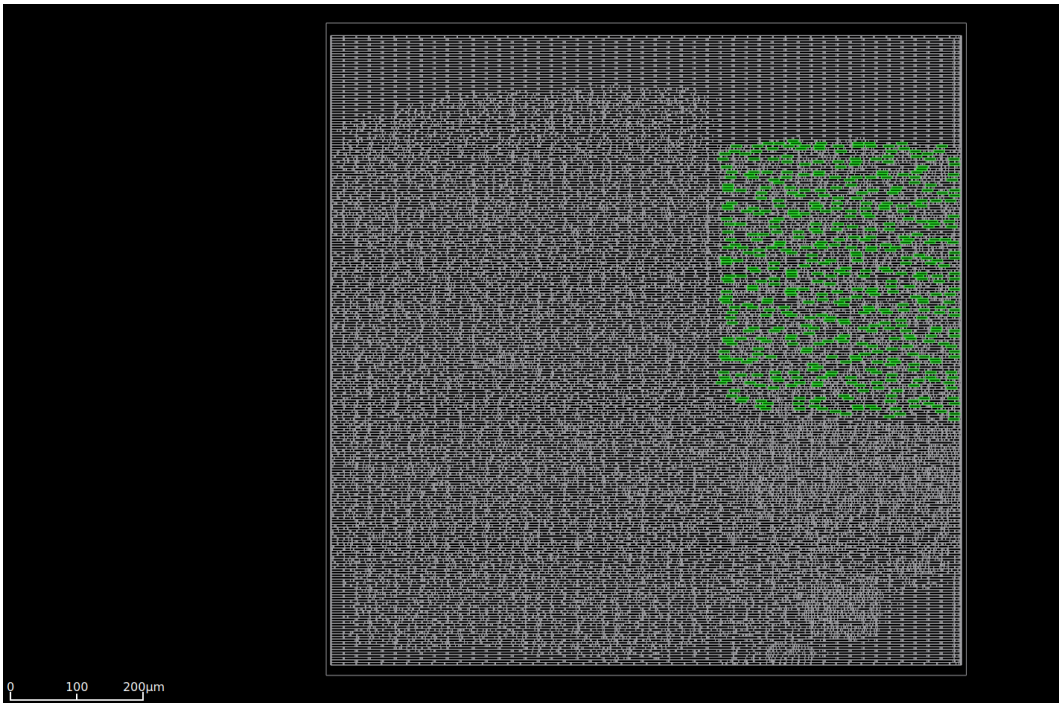


Figura 1: Identificación de los 480 flip-flops del banco de registros en el layout físico final del ASIC, utilizando OpenROAD.

Fuente: Elaboración propia.

En la Figura 1 se presenta el layout físico final del ASIC visualizado mediante OpenROAD. Las celdas resaltadas en color verde corresponden a los 480 flip-flops identificados como parte del banco de registros del procesador Espino Core, mientras que las celdas grises representan los demás elementos del circuito.

Se observa que los flip-flops del banco de registros se concentran principalmente en el

sector derecho del diseño, aunque no forman un bloque completamente uniforme. Esta distribución se debe al proceso de placement realizado por OpenROAD, que determina la ubicación de las celdas considerando las conexiones y restricciones físicas del circuito.

La identificación de las 480 instancias permite comprobar la correspondencia entre el banco de registros descrito en Verilog y los elementos de almacenamiento presentes en la implementación física final del ASIC.

4.5.2. Identificación de la ALU

La unidad aritmético-lógica, o ALU, ejecuta las operaciones aritméticas, lógicas y de comparación requeridas por el procesador.

Para identificar sus celdas fue necesario utilizar una síntesis con jerarquía preservada, debido a que las optimizaciones realizadas durante la síntesis plana dificultaban reconocer directamente los límites de este módulo.

A partir del netlist jerárquico se generó una representación aplanada que conservaba los nombres de las instancias relacionadas con la ALU.

Posteriormente, esta representación se utilizó para realizar un placement auxiliar mediante OpenROAD, independiente de la implementación física definitiva.

El procedimiento permitió identificar 589 celdas tecnológicas asociadas al módulo `espino_alu`. Todas estas instancias fueron localizadas en el placement auxiliar.

La visualización se realizó mediante los scripts ubicados en `experimentos/alu_trazabilidad/` y el archivo `scripts/resaltar_alu.tcl`.

Esta identificación corresponde al placement auxiliar generado con jerarquía preservada, por lo que no representa necesariamente la ubicación de las mismas celdas en el layout físico final del ASIC.

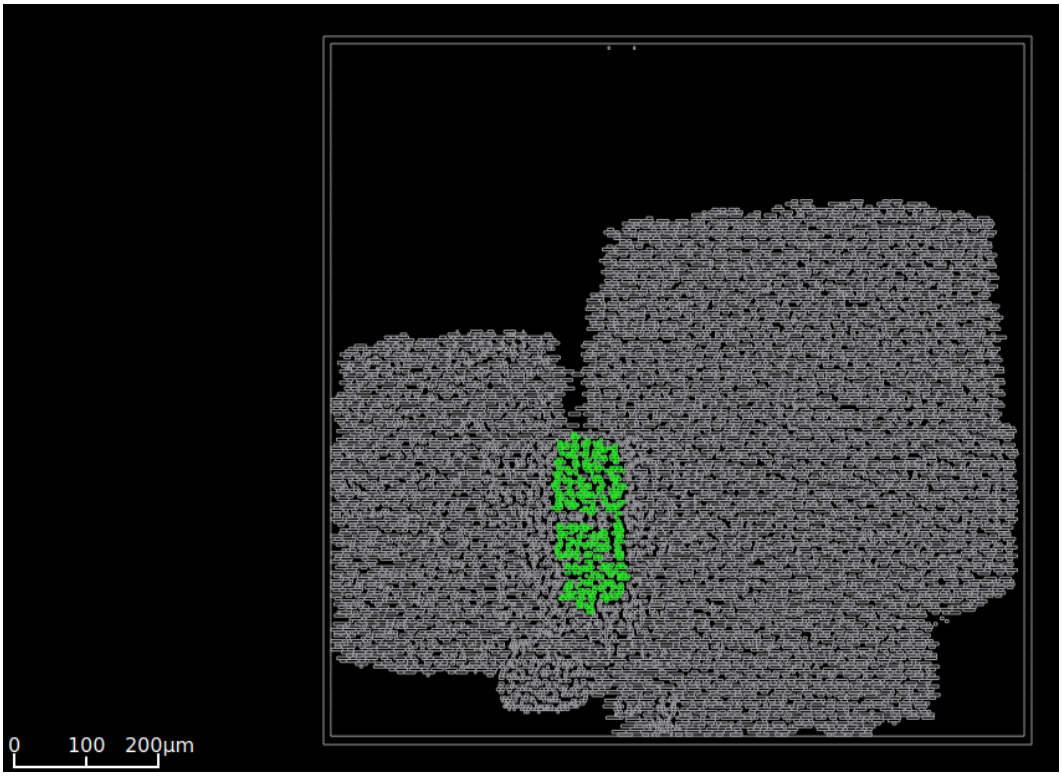


Figura 2: Identificación de las 589 celdas de la ALU mediante OpenROAD, utilizando un placement auxiliar generado a partir de una síntesis con jerarquía preservada.

Fuente: Elaboración propia.

En la Figura 2 se presenta la distribución física de las 589 celdas tecnológicas identificadas como parte de la unidad aritmético-lógica (ALU) del procesador Espino Core. Las celdas resaltadas en verde corresponden a la lógica asociada a este módulo, mientras que las celdas grises representan los demás componentes del diseño.

Se observa que las celdas de la ALU se concentran en una región cercana al centro del circuito, formando una agrupación relativamente compacta. Esta distribución es resultado del proceso de placement realizado por OpenROAD, considerando las conexiones entre las distintas celdas del diseño.

Es importante señalar que esta visualización corresponde a una implementación auxiliar obtenida mediante síntesis con jerarquía preservada. Por lo tanto, su objetivo es demostrar la correspondencia entre el módulo `espino_alu.v` y las celdas tecnológicas resultantes, sin representar necesariamente su ubicación en el layout físico definitivo del ASIC.

4.5.3. Identificación del contador de ciclos

El contador de ciclos se encuentra implementado dentro del módulo de periféricos `pochoco_periph.v`.

Mediante el análisis del netlist físico final se identificaron 32 flip-flops asociados al almacenamiento del valor del contador.

Para visualizar estas instancias se utilizó el script `scripts/resaltar_contador.tcl`, que permite resaltarlas directamente en OpenROAD.

Las instancias identificadas corresponden a los elementos `_19748_` hasta `_19779_`, verificándose la presencia de los 32 flip-flops esperados.

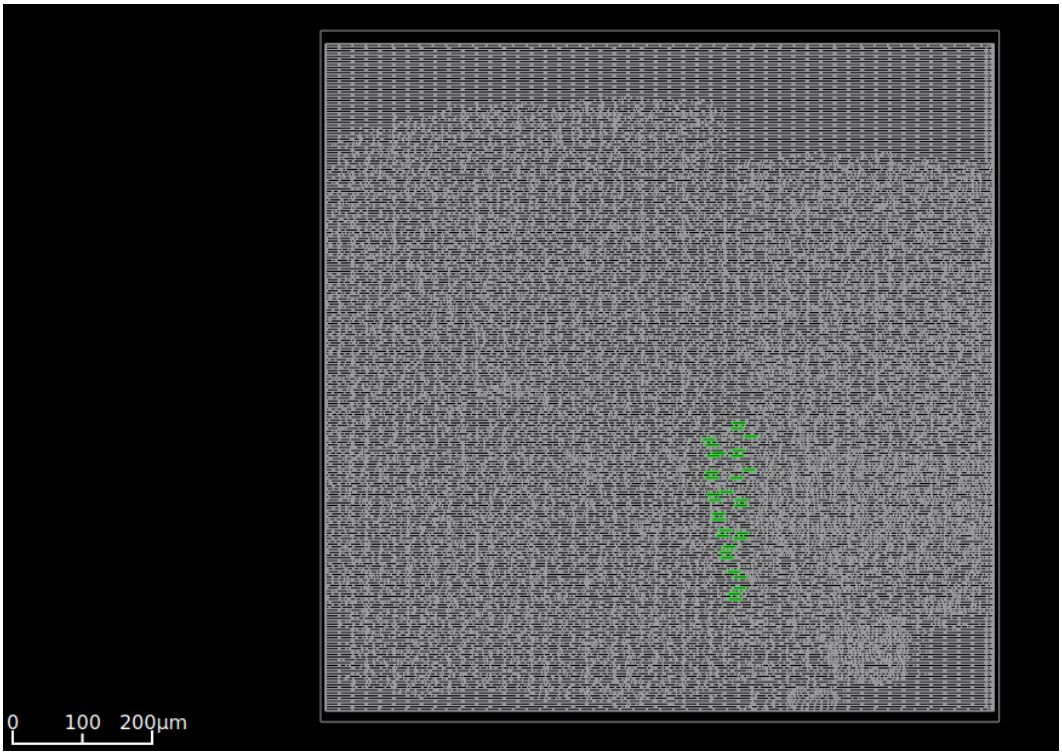


Figura 3: Identificación de los 32 flip-flops del contador de ciclos en el layout físico final del ASIC, utilizando OpenROAD.

Fuente: Elaboración propia.

En la Figura 3 se presenta la ubicación de los 32 flip-flops correspondientes al contador de ciclos dentro del layout físico final del ASIC. Las celdas resaltadas en verde representan los elementos de almacenamiento del contador, mientras que las celdas grises corresponden al resto de los componentes del circuito.

Se observa que los flip-flops del contador se encuentran distribuidos en una región relativamente concentrada del sector central del diseño. Esta disposición es resultado del proceso de placement realizado por OpenROAD, que determina la ubicación de las celdas considerando las conexiones internas y las restricciones físicas del circuito.

La identificación permite comprobar la presencia de los 32 elementos secuenciales encargados de almacenar el valor del contador. La lógica combinacional utilizada para incrementar su valor no se encuentra resaltada en esta visualización.

En conjunto, estos procedimientos permiten establecer una relación entre los módulos definidos en Verilog y su implementación mediante celdas estándar.

El banco de registros y el contador se identificaron directamente en el layout definitivo, mientras que la ALU se estudió utilizando un placement auxiliar generado a partir de una síntesis con jerarquía preservada.

5. Análisis temporal (Timing)

5.1. Configuración del análisis temporal

El análisis temporal del ASIC se realizó mediante LibreLane y OpenROAD, utilizando la tecnología GF180MCU y la biblioteca de celdas estándar `gf180mcu_fd_sc_mcu7t5v0`.

El objetivo fue comprobar que las rutas de propagación de señales cumplieran las restricciones temporales establecidas para el reloj del sistema, considerando los retardos introducidos por las celdas tecnológicas y las interconexiones físicas.

Para la implementación principal se estableció un período de reloj de 100 ns, equivalente a una frecuencia de 10 MHz. La relación entre el período y la frecuencia se expresa mediante:

$$f = \frac{1}{T} \tag{12}$$

Por lo tanto:

$$f = \frac{1}{100 \times 10^{-9}} \tag{13}$$

$$\boxed{f = 10 \text{ MHz}} \tag{14}$$

La evaluación se realizó considerando distintas esquinas de proceso, voltaje y temperatura (PVT), con el propósito de comprobar el cumplimiento temporal bajo diferentes condiciones de operación.

Las principales métricas utilizadas fueron el *Worst Negative Slack* (WNS) y el *Total Negative Slack* (TNS).

El WNS corresponde al menor margen temporal obtenido entre las rutas analizadas. Un valor positivo indica que la ruta más exigente cumple la restricción temporal, mientras que un valor negativo identifica una violación.

Por su parte, el TNS representa la suma de los márgenes negativos de las rutas que presentan violaciones. Un TNS igual a cero indica que no se detectaron violaciones de setup en las rutas evaluadas.

Adicionalmente, se consideraron los análisis de setup y hold, necesarios para verificar las condiciones temporales de funcionamiento de los elementos secuenciales del circuito.

5.2. Barrido del período de reloj

Con el objetivo de determinar el comportamiento temporal del ASIC a diferentes frecuencias, se realizaron implementaciones físicas modificando el período de reloj objetivo.

Se evaluaron períodos de 100, 70, 69, 67 y 50 ns, utilizando la arquitectura basada en una ROM combinacional y una RAM de datos reducida.

La frecuencia correspondiente a cada período se determinó mediante:

$$f_{\text{MHz}} = \frac{1000}{T_{\text{ns}}} \tag{15}$$

La Tabla 7 presenta los resultados obtenidos a partir de las métricas finales de LibreLane.

Tabla 7: Resultados del barrido de períodos de reloj.

Período (ns)	**Frecuencia (MHz)**	**WNS (ns)**	**TNS (ns)**	**Resultado**
100	10,00	+28,588	0	Cumple
70	14,29	+2,503	0	Cumple
69	14,49	-2,412	-214,467	No cumple
67	14,93	-4,412	-1347,923	No cumple
50	20,00	-13,920	-16830,236	No cumple

Fuente: Elaboración propia.

Los resultados muestran que el diseño cumplió las restricciones temporales con períodos de 100 y 70 ns, obteniéndose valores positivos de WNS y un TNS igual a cero.

Para la implementación definitiva de 100 ns se obtuvo un WNS de +28,588 ns, mientras que la implementación de 70 ns presentó un margen menor, de +2,503 ns.

Al reducir el período a 69 ns se obtuvo un WNS de -2,412 ns y un TNS de -214,467 ns, indicando la aparición de violaciones de setup.

Estas violaciones aumentaron al utilizar períodos de 67 y 50 ns, alcanzándose un WNS de -13,920 ns y un TNS de -16830,236 ns para el período más exigente.

Por lo tanto, el menor período de reloj verificado que cumplió las restricciones temporales fue de 70 ns.

La frecuencia correspondiente es:

$$f_{\text{verificada}} = \frac{1}{70 \times 10^{-9}} \tag{16}$$

$$\boxed{f_{\text{verificada}} \approx 14,29 \text{ MHz}} \tag{17}$$

Este valor representa la mayor frecuencia comprobada experimentalmente dentro de las corridas realizadas y no necesariamente la frecuencia máxima absoluta que podría alcanzar el circuito mediante modificaciones adicionales.

Para la implementación física definitiva se mantuvo un período de 100 ns, proporcionando un mayor margen temporal y conservando la correspondencia con las constantes de temporización utilizadas por el programa Assembly.

5.3. Análisis de las esquinas temporales

El análisis temporal se realizó considerando distintas esquinas PVT, que permiten evaluar el comportamiento del circuito frente a variaciones del proceso de fabricación, el voltaje de alimentación y la temperatura.

Las esquinas de proceso incluyen condiciones típicas (`tt`), lentas (`ss`) y rápidas (`ff`), utilizadas para representar variaciones en las características eléctricas de los transistores.

Para la implementación definitiva con un período de reloj de 100 ns se obtuvieron los resultados de setup presentados en la Tabla 8.

Tabla 8: Resultados de setup en distintas esquinas PVT para la implementación final de 100 ns.

Esquina PVT	**WNS (ns)**
`nom_tt_025C_5v00`	+62,576
`nom_ss_125C_4v50`	+31,400
`nom_ff_n40C_5v50`	+74,743
`max_ss_125C_4v50`	+28,588

Fuente: Elaboración propia.

La condición más desfavorable identificada corresponde a la esquina `max_ss_125C_4v50`, con un WNS de +28,588 ns.

Esta condición considera un modelo de proceso lento, una temperatura elevada y un voltaje de alimentación reducido, lo que produce una mayor exigencia sobre los tiempos de propagación del circuito.

A pesar de estas condiciones, el margen temporal obtenido fue positivo, confirmando el cumplimiento de las restricciones de setup para la implementación final.

Adicionalmente, se obtuvo un margen mínimo de hold positivo de aproximadamente +0,956 ns.

Esto indica que la implementación definitiva cumple las restricciones temporales de setup y hold evaluadas, tanto para el almacenamiento correcto de los datos como para evitar cambios prematuros en las entradas de los elementos secuenciales.

5.4. Identificación de la ruta crítica

Para identificar la ruta crítica del ASIC se analizaron los reportes de *Static Timing Analysis* (STA) generados por OpenROAD después de las etapas de placement y routing.

El análisis se realizó sobre la implementación física definitiva, correspondiente a la corrida `fisico_rom_100ns_actualizado`, utilizando la esquina temporal más desfavorable `max_ss_125C_4v50`.

El reporte identificó como ruta crítica de setup el camino comprendido entre los registros `_16974_` y `_18821_`, ambos sincronizados mediante el reloj `i_Clk`.

El registro de origen entrega la señal `u_pochoco.instr_rdata[20]`, correspondiente al bit 20 de la instrucción recibida por el procesador.

Durante su propagación, la señal atraviesa diferentes celdas combinacionales, incluyendo compuertas NAND, NOR, AOI y OAI, además de buffers utilizados para distribuir las señales dentro del circuito.

La Tabla 9 resume los resultados obtenidos directamente del reporte temporal.

Tabla 9: Características de la ruta crítica de setup del ASIC final.

Parámetro	**Resultado**
Esquina PVT	`max_ss_125C_4v50`
Registro inicial	`_16974_`
Registro final	`_18821_`
Señal de origen	`instr_rdata[20]`
Período de reloj	100 ns
Tiempo de llegada	74,449837 ns
Tiempo requerido	103,038292 ns
Slack	+28,588453 ns
Resultado	Cumple setup

Fuente: Elaboración propia.

El margen temporal de una ruta de setup se determina mediante la diferencia entre el

tiempo requerido y el tiempo de llegada de los datos:

$$\text{Slack} = t_{\text{required}} - t_{\text{arrival}} \tag{18}$$

Reemplazando los valores obtenidos:

$$\text{Slack} = 103,038292 - 74,449837 \tag{19}$$

$$\boxed{\text{Slack} \approx +28,588 \text{ ns}} \tag{20}$$

El valor positivo confirma que la ruta crítica cumple la restricción de setup establecida para un período de reloj de 100 ns.

El tiempo requerido no coincide exactamente con el período de reloj configurado, debido a que el análisis incorpora los efectos de la distribución física del reloj, la incertidumbre temporal y el tiempo de setup del elemento secuencial de destino.

El reporte también mostró otros caminos con márgenes temporales cercanos, que comparten el registro de origen `_16974_`, pero terminan en diferentes registros.

Esto permite observar que la propagación de las señales provenientes de la memoria de instrucciones a través de la lógica combinacional del procesador es relevante para el comportamiento temporal del circuito.

No obstante, la operación específica asociada al registro de destino no fue identificada únicamente a partir de este reporte, por lo que el análisis se limita a los elementos y señales comprobados.

5.5. Optimización del diseño

Con el objetivo de mejorar el comportamiento temporal y reducir el área del ASIC, se evaluó una modificación en la descripción RTL de la memoria de datos `pochoco_data_ram.v`.

En la implementación original, la escritura se realiza mediante una dirección variable que selecciona directamente la palabra de memoria correspondiente.

En la versión experimental se reemplazó esta estructura por un bloque `generate`, que implementa una lógica de habilitación de escritura independiente para cada una de las 64 palabras de memoria.

Cada palabra dispone de una señal de habilitación que combina la solicitud de acceso, la operación de escritura y la comparación de la dirección seleccionada.

Se mantiene además la habilitación individual de los cuatro bytes que componen cada

palabra de 32 bits.

El objetivo de esta modificación fue evaluar si una descripción más explícita de la decodificación de direcciones permitía mejorar los resultados de síntesis y reducir los retardos del circuito.

Para comprobar su efecto se realizaron dos implementaciones físicas con un período de reloj de 70 ns, utilizando la arquitectura original y la versión modificada.

La Tabla 10 presenta los resultados obtenidos a partir de las métricas finales de Libre-Lane.

Tabla 10: Comparación entre la RAM original y la optimizada para un período de reloj de 70 ns.

Parámetro	**Original**	**Optimizada**
Período de reloj (ns)	70	70
WNS de setup (ns)	+2,503	+0,049
TNS de setup (ns)	0	0
Área de celdas (μm^2)	463479	463001
Celdas estándar	19936	19393
Flip-flops	2859	2859
Buffers de reparación temporal	1789	1847
Cumplimiento temporal	Sí	Sí

Fuente: Elaboración propia.

Los resultados muestran que la implementación optimizada redujo el área de celdas estándar desde 463479 hasta 463001 μm^2, equivalente a una disminución de 478 μm^2.

La reducción porcentual se calcula mediante:

$$\Delta A = \frac{A_{\text{original}} - A_{\text{optimizada}}}{A_{\text{original}}} \times 100 \tag{21}$$

Reemplazando los valores:

$$\Delta A = \frac{463479 - 463001}{463479} \times 100 \tag{22}$$

$$\boxed{\Delta A \approx 0,103\,\%} \tag{23}$$

Asimismo, la cantidad de celdas estándar disminuyó de 19936 a 19393, lo que representa una reducción de 543 celdas, equivalente aproximadamente al 2,72 %.

Sin embargo, el comportamiento temporal presentó un resultado desfavorable. El WNS

disminuyó desde +2,503 ns hasta +0,049 ns.

La reducción del margen temporal fue:

$$\Delta WNS = 2,503 - 0,049 \tag{24}$$

$$\boxed{\Delta WNS \approx 2,454 \text{ ns}} \tag{25}$$

Aunque ambas implementaciones cumplen las restricciones temporales de setup, la versión modificada presenta un margen considerablemente menor.

También se observa que la cantidad de buffers utilizados para reparación temporal aumentó de 1789 a 1847, lo que indica que el flujo físico requirió más elementos de este tipo en la implementación modificada.

Por otra parte, ambas versiones conservaron 2859 celdas secuenciales, por lo que la modificación no produjo una reducción en la cantidad de elementos de almacenamiento.

En consecuencia, la optimización permitió una disminución limitada del área y del número de celdas, pero a costa de un deterioro del margen temporal.

Por este motivo, no se adoptó esta modificación para la implementación física definitiva. Se mantuvo la arquitectura original de la RAM y un período de reloj de 100 ns, priorizando el cumplimiento temporal con un margen de seguridad mayor.

5.6. Temporización del juego en Assembly

El programa del juego de reflejos utiliza un contador de ciclos de 32 bits para implementar la espera inicial de cada ronda y medir el tiempo de reacción del jugador.

Este contador incrementa su valor una vez por cada ciclo de reloj y puede ser leído mediante la dirección de memoria mapeada `0x8000000C`.

Para implementar una espera determinada se utiliza la relación:

$$N_{\text{ciclos}} = f_{\text{reloj}} \cdot t \tag{26}$$

En la implementación principal se estableció una frecuencia de reloj de 10 MHz, correspondiente a un período de 100 ns.

5.6.1. Espera inicial de tres segundos

El juego requiere mantener encendidos los cuatro LEDs durante una espera inicial de aproximadamente tres segundos antes de seleccionar el LED objetivo.

Para esta espera, el número de ciclos correspondiente es:

$$N_{\text{espera}} = 10\,000\,000 \cdot 3 \tag{27}$$

$$\boxed{N_{\text{espera}} = 30\,000\,000 \text{ ciclos}} \tag{28}$$

La constante se construye en Assembly mediante las instrucciones:

```
lui  x5, 0x1C9C
addi x5, x5, 896
```

El valor resultante es:

$$N_{\text{espera}} = \texttt{0x1C9C000} + 896 = 30\,000\,000 \tag{29}$$

El programa compara periódicamente el contador de ciclos con este umbral hasta completar la espera establecida.

5.6.2. Resolución temporal de 0,1 segundos

Para expresar el tiempo de reacción en décimas de segundo se utiliza una constante equivalente a 0,1 segundos de funcionamiento del reloj.

Considerando una frecuencia de 10 MHz:

$$N_{0,1\text{s}} = 10\,000\,000 \cdot 0,1 \tag{30}$$

$$\boxed{N_{0,1\text{s}} = 1\,000\,000 \text{ ciclos}} \tag{31}$$

Esta constante se genera mediante:

```
lui  x5, 0xF4
addi x5, x5, 576
```

El valor obtenido corresponde a:

$$N_{0,1\text{s}} = \texttt{0xF4000} + 576 = 1\,000\,000 \tag{32}$$

La conversión del tiempo de reacción utiliza este valor como referencia para determinar la cantidad de décimas de segundo transcurridas.

Es importante considerar que estas constantes dependen de la frecuencia de reloj utilizada.

Por ejemplo, si el circuito funcionara a 14,29 MHz sin modificar el programa Assembly, la duración real de las esperas sería diferente.

Por esta razón, aunque se verificó el cumplimiento temporal del ASIC con un período de 70 ns, la implementación principal se mantuvo configurada a 100 ns, conservando la correspondencia entre las constantes utilizadas por el programa y los tiempos esperados.

6. Floorplan, utilización y costos físicos

6.1. Configuración del floorplan

El floorplan corresponde a una de las primeras etapas de la implementación física de un ASIC. Su objetivo es definir las regiones disponibles para la ubicación de las celdas estándar, la distribución de las conexiones y la organización general del circuito dentro del chip.

Para el proyecto Pochoco SoC se utilizó el flujo de implementación física de LibreLane, empleando OpenROAD y la tecnología GF180MCU, con la biblioteca de celdas estándar `gf180mcu_fd_sc_mcu7t5v0`.

La implementación definitiva corresponde a la corrida `fisico_rom_100ns_actualizado`, configurada con un período de reloj de 100 ns y una utilización objetivo inicial del 40 %.

Esta configuración permitió establecer una región de implementación con espacio disponible para las celdas del circuito y para las modificaciones necesarias durante las etapas posteriores de placement, optimización temporal y routing.

Es importante distinguir dos regiones principales del diseño físico:

- **Die:** corresponde al área total definida para el chip dentro de la implementación física.
- **Core:** corresponde a la región interior donde se realiza principalmente la ubicación de las celdas estándar.

La región del core debe disponer de espacio suficiente para ubicar las celdas y permitir la distribución de sus conexiones. Por ello, no resulta conveniente ocupar la totalidad de su superficie con celdas estándar.

6.2. Área física y utilización del diseño

Después de completar el flujo de implementación física se obtuvieron las métricas finales de área presentadas en la Tabla 11.

Tabla 11: Resultados físicos de la implementación definitiva del ASIC.

Parámetro	**Resultado**
Área de celdas estándar	461844 μm^2
Área del core	902574 μm^2
Área del die	949141 μm^2
Utilización objetivo inicial	40 %
Utilización física final	51,17 %
Período de reloj	100 ns

Fuente: Elaboración propia.

La utilización física representa la relación entre el área ocupada por las celdas estándar y el área disponible dentro del core.

Se calcula mediante:

$$U = \frac{A_{\text{celdas}}}{A_{\text{core}}} \times 100 \tag{33}$$

Reemplazando los valores obtenidos:

$$U = \frac{461844}{902574} \times 100 \tag{34}$$

$$\boxed{U \approx 51,17\,\%} \tag{35}$$

Esto significa que el área total de las celdas estándar representa aproximadamente el 51,17 % de la superficie del core.

La diferencia entre el área del core y el área ocupada por las celdas es:

$$A_{\text{restante}} = A_{\text{core}} - A_{\text{celdas}} \tag{36}$$

$$A_{\text{restante}} = 902574 - 461844 \tag{37}$$

$$\boxed{A_{\text{restante}} = 440730\ \mu\text{m}^2} \tag{38}$$

Esta superficie representa aproximadamente el 48,83 % del área del core y no corresponde a un único bloque vacío, sino al espacio no contabilizado como área de celdas estándar.

Parte de este espacio es necesario para distribuir las celdas, permitir las interconexiones y satisfacer las restricciones del proceso de implementación física.

La utilización final fue superior al objetivo inicial del 40 %, debido a que ambos valores corresponden a etapas y condiciones distintas del flujo físico. Durante el proceso pueden incorporarse buffers, realizarse modificaciones de tamaño y agregarse otros elementos necesarios para cumplir las restricciones de diseño.

Por lo tanto, la utilización final debe evaluarse a partir de las métricas del circuito implementado y no únicamente del objetivo definido para el floorplan.

6.3. Distribución física de las celdas

Después de definir el floorplan, OpenROAD realizó la ubicación de las celdas estándar mediante el proceso de placement.

En esta etapa se determina la posición de las instancias tecnológicas dentro del core, considerando sus interconexiones, las restricciones de ubicación y los objetivos de implementación.

Posteriormente se realiza la distribución del reloj mediante Clock Tree Synthesis (CTS), seguida de las etapas de routing y optimización física.

Durante estas operaciones pueden incorporarse celdas adicionales, especialmente buffers, con el objetivo de mejorar la distribución de las señales y satisfacer las restricciones temporales.

La implementación final contiene diferentes tipos de celdas, entre las que se encuentran flip-flops, multiplexores, compuertas lógicas, inversores y buffers.

En particular, se identificaron 2851 flip-flops, utilizados para implementar elementos de almacenamiento del procesador, memorias y periféricos.

La distribución física de estos componentes no necesariamente coincide con la organización de módulos descrita en Verilog. Esto se debe a que el proceso de síntesis puede aplanar la jerarquía y el placement determina la ubicación de las celdas de acuerdo con criterios físicos.

Por esta razón, elementos pertenecientes a un mismo módulo RTL pueden encontrarse distribuidos en distintas regiones del circuito.

La visualización mediante OpenROAD permite observar esta distribución y relacionar la arquitectura lógica del procesador con su implementación física.

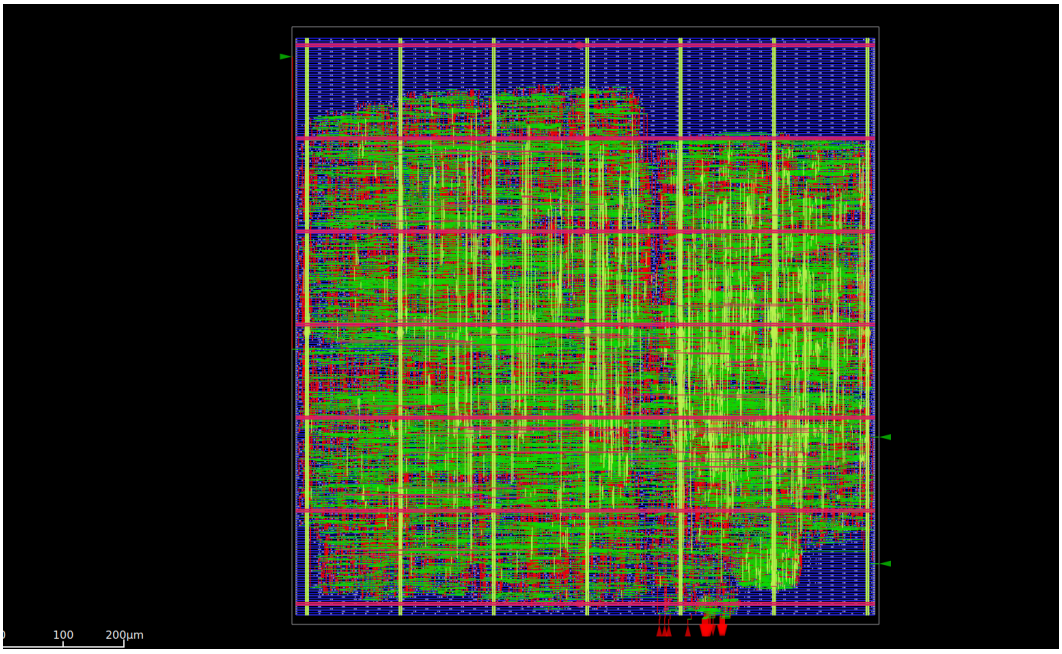


Figura 4: Layout físico final del ASIC Pochoco SoC, mostrando las interconexiones metálicas después del proceso de Place & Route.

Fuente: Elaboración propia.

En la Figura 4 se presenta el layout físico final del ASIC Pochoco SoC, obtenido mediante OpenROAD después de completar las etapas de placement, Clock Tree Synthesis y routing.

Se observa la distribución de las interconexiones metálicas dentro del área del chip. Los diferentes colores representan las capas de metal utilizadas para establecer las conexiones eléctricas entre los componentes del circuito.

También se distinguen líneas de distribución que atraviesan gran parte del diseño, junto con una red de conexiones más densa en las regiones que contienen la lógica del procesador, las memorias y los periféricos.

Esta visualización evidencia el resultado de la implementación física y permite relacionar la organización del circuito con las métricas finales de área y utilización obtenidas mediante LibreLane.

6.4. Costo físico de la implementación de memoria

Uno de los aspectos más relevantes del diseño fue la implementación de las memorias mediante celdas estándar.

En la tecnología y configuración utilizadas, las memorias descritas en RTL se implementaron utilizando elementos secuenciales y lógica combinacional, en lugar de utilizar macros SRAM dedicadas.

Esto tiene consecuencias importantes sobre el área física del ASIC, ya que cada bit almacenado requiere recursos de almacenamiento y lógica adicional para realizar las operaciones de lectura y escritura.

6.4.1. Memoria de datos de la arquitectura principal

La arquitectura principal utiliza una memoria RAM de datos de 64 palabras, con un ancho de 32 bits por palabra.

La capacidad total se calcula mediante:

$$C_{\text{RAM}} = N_{\text{palabras}} \times N_{\text{bits/palabra}} \tag{39}$$

Por lo tanto:

$$C_{\text{RAM}} = 64 \times 32 \tag{40}$$

$$\boxed{C_{\text{RAM}} = 2048 \text{ bits}} \tag{41}$$

Esto equivale a:

$$C_{\text{RAM}} = \frac{2048}{8} = \boxed{256 \text{ bytes}} \tag{42}$$

Una implementación basada en registros requiere almacenamiento para estos 2048 bits, además de la lógica necesaria para seleccionar direcciones, habilitar escrituras y entregar los datos de lectura.

Debido a ello, el costo físico de la memoria no depende únicamente de la cantidad de bits almacenados, sino también de la estructura de acceso implementada durante la síntesis.

6.4.2. Memoria de la arquitectura unificada

Durante el desarrollo también se evaluó una arquitectura alternativa con una memoria unificada de 128 palabras de 32 bits.

Su capacidad es:

$$C_{\text{unificada}} = 128 \times 32 \tag{43}$$

$$\boxed{C_{\text{unificada}} = 4096 \text{ bits}} \tag{44}$$

Equivalente a:

$$C_{\text{unificada}} = \frac{4096}{8} = \boxed{512 \text{ bytes}} \tag{45}$$

La memoria unificada dispone del doble de capacidad de almacenamiento que la RAM de datos de la arquitectura principal.

Sin embargo, ambas arquitecturas no presentan exactamente la misma función: la implementación principal utiliza una ROM combinacional para las instrucciones y una RAM separada para los datos, mientras que la variante unificada considera una organización de memoria diferente.

Por este motivo, la comparación de costos debe realizarse considerando el diseño completo y no únicamente la capacidad de las memorias.

6.5. Comparación del costo físico entre arquitecturas

Para evaluar el impacto de la arquitectura de memoria se compararon las métricas físicas de la implementación principal y de la versión experimental con memoria unificada.

Ambas alternativas se analizaron con un período de reloj objetivo de 100 ns.

La Tabla 12 resume los resultados obtenidos.

Tabla 12: Comparación física entre la arquitectura principal y la memoria unificada.

Parámetro	**ROM + RAM**	**Unificada**
Área de celdas (μm^2)	461844	877588
Flip-flops	2851	4914
Utilización final	51,17 %	52,01 %
Período (ns)	100	100
WNS de setup (ns)	+28,588	-0,319
Cumple setup	Sí	No

Fuente: Elaboración propia.

Los resultados muestran que la implementación con memoria unificada presenta un área de celdas estándar considerablemente mayor.

La diferencia de área es:

$$\Delta A = A_{\text{unificada}} - A_{\text{principal}} \tag{46}$$

$$\Delta A = 877588 - 461844 \tag{47}$$

$$\boxed{\Delta A = 415744\ \mu\text{m}^2} \tag{48}$$

El aumento porcentual respecto de la arquitectura principal es:

$$\Delta A_{\,\%} = \frac{415744}{461844} \times 100 \tag{49}$$

$$\boxed{\Delta A_{\,\%} \approx 90,02\,\%} \tag{50}$$

Por lo tanto, la arquitectura con memoria unificada requirió aproximadamente un 90 % más de área de celdas estándar.

También se observó un aumento en la cantidad de flip-flops:

$$\Delta N_{\text{FF}} = 4914 - 2851 \tag{51}$$

$$\boxed{\Delta N_{\text{FF}} = 2063} \tag{52}$$

Este aumento es consistente con el mayor almacenamiento secuencial requerido por la variante unificada.

Sin embargo, no debe atribuirse toda la diferencia exclusivamente a los bits de memoria, ya que las arquitecturas también pueden diferir en la lógica de acceso, selección y control, además de los elementos añadidos durante la implementación física.

Un resultado importante es que ambas arquitecturas presentan porcentajes de utilización similares, cercanos al 52 %, pese a que sus áreas de celdas son muy diferentes.

Esto se explica porque la utilización es una relación entre el área de las celdas y el área del core, por lo que no representa directamente el tamaño absoluto del circuito.

Desde el punto de vista temporal, la arquitectura principal obtuvo un WNS de +28,588 ns para un período de 100 ns, mientras que la versión unificada presentó un WNS de -0,319 ns.

Por lo tanto, la implementación principal cumplió las restricciones temporales, mientras que la alternativa unificada presentó una violación de setup bajo las condiciones

evaluadas.

En consecuencia, la arquitectura basada en ROM combinacional y RAM de datos reducida resultó más conveniente para la implementación definitiva, debido a su menor área física y mejor margen temporal.

Estos resultados muestran que la selección de la arquitectura de memoria tiene un impacto significativo sobre la cantidad de recursos tecnológicos necesarios y sobre la viabilidad física del ASIC.

7. Verificación física y signoff

7.1. Objetivo de la verificación física

Una vez completadas las etapas de síntesis, floorplan, placement, Clock Tree Synthesis (CTS) y routing, es necesario verificar que la implementación física del ASIC cumpla las restricciones eléctricas, geométricas y temporales establecidas para la tecnología utilizada.

Este conjunto de comprobaciones se conoce como *signoff* y constituye una etapa fundamental del flujo de diseño de circuitos integrados.

Para el proyecto Pochoco SoC se utilizaron las herramientas integradas en LibreLane, trabajando con la tecnología GF180MCU y la biblioteca de celdas estándar `gf180mcu_fd_sc_mcu7t5v0`.

La implementación analizada corresponde a la corrida `fisico_rom_100ns_actualizado`, configurada con un período de reloj de 100 ns.

Las principales verificaciones realizadas fueron:

- **DRC (Design Rule Check):** comprobación del cumplimiento de las reglas geométricas de fabricación.
- **LVS (Layout Versus Schematic):** comparación entre la conectividad del layout y la descripción eléctrica de referencia.
- **Verificación de antenas:** detección de posibles problemas asociados a las interconexiones metálicas durante el proceso de fabricación.
- **Análisis temporal de setup y hold:** comprobación del cumplimiento de las restricciones temporales de los elementos secuenciales.

El objetivo de estas verificaciones fue determinar si el diseño implementado presentaba errores físicos o violaciones temporales que impidieran considerar satisfactoria la implementación obtenida.

7.2. Verificación DRC

El *Design Rule Check* (DRC) permite comprobar que las geometrías presentes en el layout respeten las restricciones establecidas por el proceso de fabricación.

Estas restricciones incluyen aspectos como el ancho mínimo de las pistas, la separación entre elementos, las dimensiones de las vías y las relaciones geométricas entre diferentes capas del circuito.

Su cumplimiento es importante para evitar problemas que puedan afectar la fabricación y el funcionamiento del ASIC.

En el proyecto se utilizó la verificación DRC realizada mediante Magic, integrada en el flujo de LibreLane.

El resultado obtenido para la implementación física definitiva fue:

$$\boxed{N_{\text{DRC}} = 0} \tag{53}$$

Esto significa que la comprobación realizada mediante Magic no detectó violaciones de las reglas de diseño evaluadas.

Es importante señalar que este resultado corresponde a la verificación ejecutada con Magic. La comprobación DRC independiente mediante KLayout fue omitida dentro del flujo, por lo que no se dispone de un resultado adicional de esta herramienta.

Por lo tanto, el cumplimiento reportado se limita al conjunto de reglas comprobadas mediante Magic, sin representar una certificación externa completa de fabricación.

7.3. Verificación LVS

La verificación *Layout Versus Schematic* (LVS) permite comprobar que la conectividad eléctrica extraída del layout corresponda con la descripción del circuito utilizada como referencia.

Mientras que el DRC analiza principalmente las características geométricas, el LVS se concentra en la correspondencia de las conexiones y los dispositivos presentes en la implementación.

Para realizar esta comprobación se compara una representación eléctrica obtenida del layout con el netlist de referencia correspondiente al diseño.

Entre los problemas que pueden detectarse se encuentran conexiones faltantes, conexiones adicionales o diferencias entre los elementos del circuito.

En la implementación final de Pochoco SoC, la verificación LVS terminó satisfactoriamente, sin reportarse discrepancias de conectividad que impidieran aprobar la comparación.

Por lo tanto, el resultado obtenido fue:

$$\boxed{\text{LVS} = \text{PASS}} \tag{54}$$

Este resultado permite confirmar que la implementación física conserva la conectividad

eléctrica esperada de acuerdo con el netlist de referencia utilizado en la verificación.

Sin embargo, el cumplimiento de LVS no demuestra por sí mismo el comportamiento funcional del procesador, por lo que debe complementarse con las simulaciones y verificaciones temporales correspondientes.

7.4. Verificación de antenas

Durante la fabricación de un circuito integrado, determinadas estructuras metálicas pueden acumular carga eléctrica en las etapas de procesamiento.

Si esta carga se concentra en conexiones asociadas a compuertas sensibles, podría provocar daños en los dispositivos.

Este fenómeno se conoce como *efecto antena* y se controla mediante reglas que relacionan las dimensiones de las interconexiones con las características de los dispositivos conectados.

La verificación de antenas permite identificar interconexiones que podrían incumplir estas restricciones.

En el flujo físico del proyecto se ejecutó la comprobación correspondiente, obteniéndose el siguiente resultado final:

$$\boxed{N_{\text{antenas}} = 0} \tag{55}$$

Por lo tanto, no se reportaron violaciones de antena en la implementación física definitiva.

Este resultado indica que el diseño cumplió las comprobaciones de antena realizadas por el flujo de implementación utilizado.

7.5. Verificación temporal final

Además de las comprobaciones geométricas y de conectividad, se verificó el cumplimiento de las restricciones temporales después de completar la implementación física.

El análisis se realizó mediante OpenROAD, considerando los retardos de las celdas tecnológicas y las interconexiones del circuito implementado.

Las dos verificaciones principales fueron setup y hold.

7.5.1. Verificación de setup

La restricción de setup establece que los datos deben encontrarse disponibles con suficiente anticipación respecto del flanco activo del reloj que realiza su captura.

Si una señal llega demasiado tarde al registro de destino, se produce una violación de setup.

Para la implementación definitiva, configurada con un período de reloj de 100 ns, se obtuvo un Worst Negative Slack (WNS) de:

$$\boxed{WNS_{\text{setup}} = +28,588 \text{ ns}} \tag{56}$$

El valor positivo confirma que la ruta de setup más exigente cumple la restricción temporal evaluada.

El análisis de la ruta crítica, presentado en el apartado anterior, permitió identificar el camino comprendido entre los registros `_16974_` y `_18821_`.

La esquina temporal más desfavorable fue `max_ss_125C_4v50`, bajo la cual se obtuvo el margen temporal indicado.

Adicionalmente, el TNS de setup fue igual a cero, indicando que no se registraron violaciones de setup en el análisis final.

7.5.2. Verificación de hold

La restricción de hold establece que los datos deben permanecer estables durante un intervalo mínimo después del flanco activo del reloj.

Cuando una señal cambia demasiado pronto, puede producirse una violación de hold.

Para la implementación física definitiva se obtuvo un margen mínimo de hold de aproximadamente:

$$\boxed{WS_{\text{hold}} = +0,956 \text{ ns}} \tag{57}$$

El resultado positivo indica que las rutas evaluadas cumplen sus restricciones de hold.

Por lo tanto, la implementación final presentó márgenes positivos tanto para setup como para hold, confirmando el cumplimiento temporal en las condiciones analizadas.

7.6. Resumen de los resultados de signoff

La Tabla 13 presenta los principales resultados obtenidos durante las verificaciones finales del ASIC.

Tabla 13: Resultados de verificación física y temporal de la implementación final del ASIC.

Verificación	**Resultado**	**Estado**
DRC mediante Magic	0 violaciones	Cumple
LVS	Comparación exitosa	Cumple
Antenas	0 violaciones	Cumple
Setup WNS	+28,588 ns	Cumple
Setup TNS	0 ns	Cumple
Hold WS	+0,956 ns	Cumple
DRC mediante KLayout	No ejecutado	N/A

Fuente: Elaboración propia.

Los resultados muestran que la implementación definitiva completó satisfactoriamente las principales verificaciones físicas y temporales ejecutadas durante el flujo.

La comprobación DRC realizada mediante Magic no detectó violaciones geométricas, mientras que LVS confirmó la correspondencia de conectividad con el netlist de referencia.

Asimismo, la verificación de antenas finalizó sin violaciones reportadas.

Desde el punto de vista temporal, los márgenes positivos de setup y hold indican que el ASIC cumple las restricciones establecidas para un período de reloj de 100 ns.

No obstante, debe considerarse que la verificación DRC mediante KLayout no fue ejecutada, por lo que los resultados disponibles no constituyen una validación independiente mediante todas las herramientas posibles.

7.7. Conclusión de la verificación física

A partir de los resultados obtenidos, se concluye que la implementación física definitiva del ASIC Pochoco SoC completó satisfactoriamente las verificaciones ejecutadas durante el flujo de LibreLane.

El diseño presentó cero violaciones DRC según Magic, una comparación LVS exitosa y cero violaciones de antena.

Además, se verificó el cumplimiento temporal para un período de reloj de 100 ns, con

un margen de setup de +28,588 ns y un margen mínimo de hold de aproximadamente +0,956 ns.

Estos resultados permiten considerar satisfactoria la implementación física obtenida dentro del entorno experimental y las condiciones de verificación utilizadas.

Sin embargo, el cumplimiento de estas comprobaciones no constituye por sí solo una garantía de fabricación exitosa o funcionamiento en silicio, ya que para ello serían necesarias verificaciones adicionales propias de un proceso completo de fabricación y validación.

En consecuencia, el proyecto demuestra la viabilidad de implementar la arquitectura Pochoco SoC mediante un flujo de diseño ASIC basado en herramientas abiertas, obteniendo un layout físicamente verificado mediante las comprobaciones disponibles.

8. Simulación RTL y Gate-Level

8.1. Objetivo de la verificación funcional

La verificación funcional constituye una etapa fundamental del desarrollo de un circuito integrado, ya que permite comprobar que el comportamiento del sistema se mantiene durante las distintas etapas de implementación.

En este proyecto, se realizaron simulaciones a nivel RTL (*Register Transfer Level*) y a nivel de compuertas (*gate-level*) del sistema `Pochoco SoC`, con el objetivo de evaluar la consistencia entre la descripción original en Verilog y el circuito sintetizado mediante el flujo de diseño ASIC.

La simulación RTL permite observar el funcionamiento de los módulos descritos mediante registros, operaciones lógicas y estructuras de control. Por otra parte, la simulación gate-level utiliza un netlist compuesto por celdas estándar de la tecnología seleccionada, representando la implementación lógica obtenida después de la síntesis y optimización del diseño.

Para comparar ambas representaciones, se aplicaron condiciones equivalentes de reloj, reset y entradas digitales, verificando las transiciones de las señales externas principales del sistema.

8.2. Entorno y metodología de simulación

Las simulaciones se realizaron utilizando **Icarus Verilog** como herramienta de compilación y ejecución, y **GTKWave** para la visualización de las formas de onda generadas.

Se utilizó un testbench que proporciona una señal de reloj de 10 MHz, equivalente a un período de 100 ns:

$$f_{\text{clk}} = \frac{1}{T_{\text{clk}}} = \frac{1}{100 \times 10^{-9}} = 10\,\text{MHz} \tag{58}$$

El testbench controla las señales de entrada del circuito, incluyendo el reset y los pulsadores del juego, y monitorea las salidas correspondientes a los LEDs.

La secuencia de prueba utilizada fue la siguiente:

1. Inicializar las entradas y activar el reset durante cinco ciclos de reloj.
2. Liberar el reset y permitir que el procesador comience la ejecución del programa.
3. Aplicar una pulsación de inicio mediante `switches = 0001` durante 20 ciclos.

4. Liberar el pulsador y esperar hasta observar la salida `leds = 1111`.
5. Aplicar una segunda pulsación mediante `switches = 0010` durante 20 ciclos.
6. Liberar los pulsadores y observar las salidas durante 300 ciclos adicionales.
7. Registrar el estado final de los LEDs y finalizar la simulación.

Para garantizar condiciones comparables, se utilizaron testbenches con la misma secuencia de estímulos. En el caso RTL, se configuró el parámetro `SIMULATION = 0`, correspondiente a la configuración funcional que se buscaba reproducir en el netlist tecnológico.

En ambos casos se generaron archivos VCD (*Value Change Dump*) para registrar las transiciones de las señales analizadas.

8.3. Simulación a nivel RTL

La simulación RTL se ejecutó utilizando los módulos Verilog originales del proyecto, incluyendo el procesador, las memorias, los periféricos y el módulo superior `asic_top`.

La compilación se realizó mediante Icarus Verilog, seleccionando el testbench de comparación como módulo principal y utilizando las descripciones RTL del sistema.

La compilación finalizó correctamente, con código de salida cero, permitiendo ejecutar el diseño sin errores de elaboración.

Durante la simulación se observó que los LEDs presentaron inicialmente el valor:

$$\texttt{LEDs}_{\text{inicial}} = 0000 \tag{59}$$

Después de aplicar la secuencia de inicio, la salida presentó una transición al valor:

$$\texttt{LEDs} = 1111 \tag{60}$$

Esta transición ocurrió a los:

$$t_{\text{transición}} = 13{,}65\,\mu\text{s} \tag{61}$$

Posteriormente, se aplicó una segunda pulsación y se observó el comportamiento de las salidas hasta la finalización del testbench.

El estado final registrado fue:

$$\texttt{LEDs}_{\text{final}} = 1111 \tag{62}$$

La ejecución finalizó correctamente a los 47,6 µs de tiempo simulado.

Estos resultados permiten establecer una referencia funcional para comparar el comportamiento del diseño RTL con su implementación tecnológica.

8.4. Simulación Gate-Level del ASIC

Una vez completado el flujo de síntesis e implementación física mediante LibreLane, se realizó una simulación gate-level utilizando el netlist tecnológico final del circuito.

El archivo utilizado fue:

`final/nl/asic_top.nl.v`

Este netlist contiene las instancias de celdas estándar correspondientes a la biblioteca `gf180mcu_fd_sc_mcu7t5v0`, perteneciente a la tecnología GF180MCU.

Para permitir la compilación, se incorporaron los modelos Verilog de las celdas estándar y el archivo `primitives.v`, que contiene las primitivas internas utilizadas por la biblioteca.

Debido a limitaciones de compatibilidad de Icarus Verilog con determinadas construcciones temporales de los modelos tecnológicos, la simulación funcional se compiló utilizando la opción `-gno-specify`.

Esta configuración permite evaluar el comportamiento lógico del netlist sin considerar los retardos físicos del circuito.

El testbench se adaptó para instanciar directamente el módulo `asic_top` sintetizado, sin utilizar el parámetro `SIMULATION`, ya que este no se encuentra presente en el netlist tecnológico.

La compilación gate-level finalizó correctamente, con código de salida cero.

Durante su ejecución se obtuvieron los siguientes resultados:

- Estado inicial de los LEDs: `0000`.
- Aplicación de la pulsación de inicio.
- Transición de los LEDs a `1111`.
- Instante de transición: 13,65 µs.

- Aplicación y liberación de una segunda pulsación.
- Estado final de los LEDs: `1111`.
- Finalización normal a los 47,6 µs.

La simulación gate-level terminó con código de salida cero, confirmando que el netlist pudo ejecutarse con los estímulos definidos.

8.5. Análisis de las formas de onda

Las formas de onda obtenidas fueron analizadas mediante GTKWave, utilizando los archivos VCD generados por las simulaciones.

Para facilitar su interpretación, se seleccionaron las señales externas principales:

- `clk`: reloj del sistema.
- `rst_n`: reset activo en bajo.
- `switches[3:0]`: entradas de los pulsadores.
- `leds[3:0]`: salidas de los LEDs.

La Figura 5 muestra las señales registradas durante la simulación gate-level del circuito.

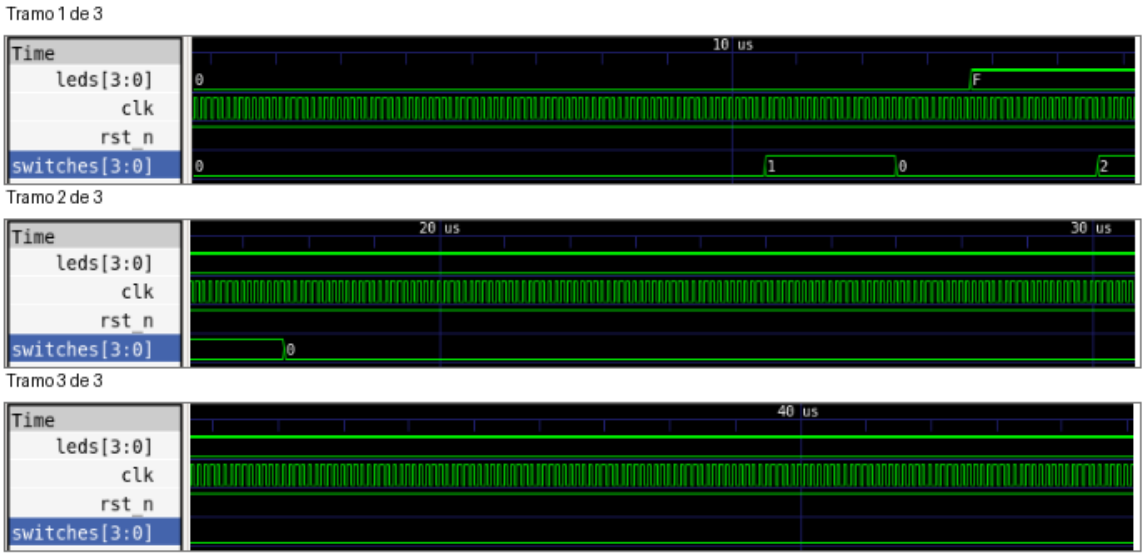


Figura 5: Formas de onda de la simulación gate-level del ASIC.

Fuente: Elaboración propia.

En la figura se observa la oscilación del reloj, la liberación de la señal de reset y las modificaciones de las entradas digitales durante la ejecución del testbench.

La salida de LEDs comienza en el estado `0000` y posteriormente presenta una transición al estado `1111`, manteniéndose en este último hasta el final de la simulación.

La transición se produce a los 13,65 µs, en concordancia con los mensajes registrados por el simulador.

8.6. Comparación funcional entre RTL y Gate-Level

Con el objetivo de evaluar la correspondencia entre ambas representaciones del circuito, se ejecutaron las simulaciones RTL y gate-level utilizando las mismas condiciones de entrada y frecuencia de reloj.

La Tabla 14 resume los resultados obtenidos.

Tabla 14: Resultados de las simulaciones RTL y gate-level.

Parámetro	**RTL**	**Gate-level**
Compilación	Correcta	Correcta
Código de ejecución	0	0
Frecuencia del reloj	10 MHz	10 MHz
LEDs iniciales	0000	0000
Transición de LEDs	1111	1111
Tiempo de transición	13,65 µs	13,65 µs
LEDs finales	1111	1111
Fin del testbench	47,6 µs	47,6 µs

Fuente: Elaboración propia.

Se observa que ambas simulaciones reprodujeron los mismos resultados para las señales de salida analizadas.

Para profundizar la comparación, se desarrolló un procedimiento automático en Python que analiza los archivos VCD generados.

Este procedimiento identifica las transiciones de las señales seleccionadas y compara tanto sus valores como los instantes en los que ocurren.

Las señales evaluadas fueron:

$$S = \{\texttt{clk}, \texttt{rst_n}, \texttt{switches}, \texttt{leds}\} \tag{63}$$

El resultado se presenta en la Tabla 15.

Tabla 15: Comparación automática de transiciones VCD.

Señal	RTL	Gate-level	Resultado
`clk`	953	953	Coinciden
`rst_n`	2	2	Coinciden
`switches`	5	5	Coinciden
`leds`	2	2	Coinciden

Fuente: Elaboración propia.

El procedimiento automático entregó el resultado:

COINCIDENCIA EXACTA

Esto indica que las cuatro señales comparadas presentaron las mismas transiciones, con valores e instantes idénticos, durante el intervalo simulado.

En particular, la salida de LEDs realizó la misma transición de `0000` a `1111` a los 13,65 µs.

La coincidencia temporal es consistente con la configuración de simulación funcional, en la cual no se incorporaron los retardos físicos de las interconexiones ni de las celdas mediante anotación SDF.

Estos resultados proporcionan evidencia de consistencia funcional entre la descripción RTL y el netlist tecnológico para la secuencia de prueba evaluada.

8.7. Alcance y limitaciones de la verificación

Aunque se obtuvo coincidencia exacta en las señales analizadas, es importante delimitar el alcance de los resultados.

La comparación realizada considera únicamente un conjunto de entradas y salidas externas, por lo que no constituye una demostración de equivalencia formal de todos los estados internos del procesador.

Asimismo, la secuencia de prueba permitió observar la inicialización, las pulsaciones aplicadas y una transición de los LEDs, pero no verificó exhaustivamente el mecanismo de selección de un LED objetivo, la evaluación de respuestas ni la totalidad de las rondas del juego.

Por otra parte, la simulación gate-level se ejecutó sin anotación de retardos SDF. En consecuencia, esta prueba comprueba el comportamiento lógico del netlist, pero no permite determinar directamente los retardos reales de propagación del circuito implementado.

La evaluación temporal de la implementación física se realizó de manera independiente mediante el análisis estático de temporización (STA) del flujo LibreLane, considerando las esquinas tecnológicas y restricciones correspondientes.

Por lo tanto, los resultados obtenidos deben interpretarse como una verificación funcional parcial del diseño sintetizado, complementaria a los análisis de síntesis, temporización e implementación física.

8.8. Conclusión de la verificación funcional

Las simulaciones realizadas permitieron comprobar que el diseño RTL y el netlist tecnológico final del ASIC presentan un comportamiento consistente para la secuencia de estímulos analizada.

Ambas representaciones finalizaron correctamente su ejecución y reprodujeron la misma transición de los LEDs a los 13,65 µs, manteniendo el estado final `1111` hasta los 47,6 µs de tiempo simulado.

Adicionalmente, la comparación automática de archivos VCD confirmó coincidencia exacta en las transiciones registradas para el reloj, reset, pulsadores y LEDs.

De esta manera, se obtuvo evidencia experimental de que la implementación tecnológica conserva el comportamiento externo observado en el RTL para las condiciones de prueba utilizadas.

Estos resultados complementan las verificaciones físicas y temporales del diseño ASIC, aunque no reemplazan una comprobación formal de equivalencia ni una validación funcional exhaustiva del sistema completo.

9. Conclusiones y trabajo futuro

9.1. Conclusiones generales

El desarrollo del presente proyecto permitió implementar un sistema digital basado en el procesador RV32E `Pochoco SoC`, adaptando una arquitectura originalmente desarrollada para FPGA hacia un flujo de diseño de circuito integrado de aplicación específica (ASIC).

La implementación se realizó utilizando la tecnología GF180MCU, mediante herramientas de síntesis lógica, implementación física y verificación integradas en LibreLane.

El sistema conserva la arquitectura principal del procesador, incluyendo la unidad de procesamiento, la memoria de instrucciones, la memoria de datos y los periféricos de entrada y salida. Además, integra un programa en lenguaje ensamblador que implementa la lógica de un juego de reacción mediante pulsadores, LEDs y visualizadores de siete segmentos.

La transición desde FPGA hacia ASIC permitió estudiar las diferencias entre ambas plataformas, especialmente en la representación de memorias, la utilización de celdas estándar, la distribución física de componentes y el cumplimiento de restricciones temporales.

9.2. Resultados de síntesis e implementación física

La implementación física definitiva se realizó con un período de reloj objetivo de 100 ns, equivalente a una frecuencia de 10 MHz.

Los resultados obtenidos muestran que el circuito pudo ser sintetizado y distribuido físicamente utilizando la biblioteca de celdas estándar `gf180mcu_fd_sc_mcu7t5v0`.

La Tabla 16 presenta un resumen de las principales características de la implementación final.

Tabla 16: Resumen de los resultados de implementación ASIC.

Parámetro	**Resultado**
Tecnología	GF180MCU
Período objetivo	100 ns
Frecuencia objetivo	10 MHz
Área de celdas estándar	461844 μm^2
Área del core	902574 μm^2
Área del die	949141 μm^2
Utilización final	51,17 %
Flip-flops	2851
Peor slack de setup	+28,588 ns
Peor slack de hold	+0,956 ns
TNS de setup	0 ns

Fuente: Elaboración propia.

El área ocupada por las celdas estándar fue de 461844 μm^2, mientras que el área total del die alcanzó 949141 μm^2.

La utilización final de aproximadamente 51,17 % indica que las celdas estándar ocupan cerca de la mitad del área correspondiente al core, dejando espacio para las interconexiones y otros elementos requeridos durante la implementación física.

La presencia de 2851 flip-flops evidencia la importancia de los recursos secuenciales en la arquitectura implementada, incluyendo el almacenamiento asociado al procesador, sus registros y las estructuras de memoria.

Estos resultados permitieron obtener una implementación física completa del sistema, incluyendo ubicación de celdas, distribución del reloj y enrutamiento de interconexiones.

9.3. Cumplimiento temporal y frecuencia de operación

El análisis estático de temporización permitió evaluar el cumplimiento de las restricciones de setup y hold bajo diferentes condiciones de operación de la tecnología.

Para la implementación definitiva de 100 ns, el peor resultado de setup se obtuvo en la esquina tecnológica `max_ss_125C_4v50`, con un slack positivo de:

$$WNS_{\text{setup}} = +28{,}588 \text{ ns} \tag{64}$$

De manera complementaria, se obtuvo un slack de hold positivo de aproximadamente:

$$WNS_{\text{hold}} = +0{,}956 \text{ ns} \tag{65}$$

Estos resultados, junto con un TNS de setup igual a cero, indican que la implementación final satisface las restricciones temporales evaluadas para el período objetivo de 100 ns.

Adicionalmente, se realizaron implementaciones con períodos de reloj más exigentes para estudiar el límite práctico de frecuencia del diseño.

La configuración de 70 ns presentó un slack de setup positivo de aproximadamente 2,503 ns, mientras que la implementación de 69 ns presentó un slack negativo de aproximadamente -2,412 ns.

Por lo tanto, el menor período de reloj verificado satisfactoriamente entre las implementaciones evaluadas fue de 70 ns, equivalente a:

$$f = \frac{1}{70 \times 10^{-9}} \approx 14{,}29 \text{ MHz} \tag{66}$$

Este valor corresponde a la mayor frecuencia verificada entre las configuraciones estudiadas, y no necesariamente a la frecuencia máxima absoluta alcanzable mediante nuevas optimizaciones físicas.

9.4. Evaluación de alternativas y optimizaciones

Durante el desarrollo se analizaron alternativas de organización de memoria y modificaciones de la descripción RTL para evaluar su impacto sobre el área y la temporización del circuito.

Una de las alternativas estudiadas correspondió a una arquitectura de memoria unificada, que reemplazaba la organización principal basada en memoria de instrucciones y memoria de datos separadas.

Sin embargo, esta alternativa presentó un aumento significativo del área de celdas estándar, alcanzando aproximadamente 877588 μm^2 frente a los 461844 μm^2 de la implementación final seleccionada.

Esto representa un incremento cercano al 90 % respecto del diseño definitivo.

Adicionalmente, la implementación de memoria unificada presentó un slack negativo de aproximadamente -0,319 ns para un período objetivo de 100 ns, por lo que no cumplió la restricción temporal evaluada.

También se estudió una modificación en la descripción RTL de la memoria de datos, buscando mejorar la forma en que sus operaciones eran representadas durante la síntesis lógica.

Para las implementaciones comparadas a 70 ns, la versión modificada redujo ligeramente el área de celdas estándar desde 463479 μm^2 hasta 463001 μm^2.

No obstante, el margen temporal disminuyó, pasando de aproximadamente +2,503 ns a +0,049 ns.

Aunque esta modificación permitió reducir el área, no proporcionó una mejora conjunta de área y temporización.

Por esta razón, se mantuvo como implementación definitiva la arquitectura con memoria de instrucciones separada y memoria de datos original.

Estos experimentos evidenciaron que una reducción de recursos lógicos no implica necesariamente un mejor resultado físico, ya que también influyen el posicionamiento de celdas, el enrutamiento y los caminos críticos de propagación.

9.5. Verificaciones físicas y funcionales

La implementación definitiva fue sometida a distintas verificaciones orientadas a comprobar la consistencia del diseño físico.

Entre los resultados obtenidos destaca la ausencia de violaciones en la verificación DRC realizada mediante Magic, así como un resultado satisfactorio en la comparación LVS entre las representaciones física y lógica del circuito.

Asimismo, la verificación de antena no reportó violaciones.

Por otra parte, el análisis estático de temporización confirmó el cumplimiento de las restricciones de setup y hold para la configuración final seleccionada.

En cuanto a la verificación funcional, se realizaron simulaciones RTL y gate-level utilizando Icarus Verilog.

La simulación gate-level empleó el netlist tecnológico final junto con los modelos Verilog de las celdas estándar GF180MCU.

Ambas representaciones se ejecutaron utilizando una frecuencia de reloj de 10 MHz y condiciones equivalentes de entrada.

En las dos simulaciones se registró una transición de la salida de LEDs desde `0000` hasta `1111` a los 13,65 μs, finalizando el testbench a los 47,6 μs.

Además, la comparación automática de los archivos VCD mostró coincidencia exacta en los valores y tiempos de transición de las señales `clk`, `rst_n`, `switches` y `leds`.

Estos resultados proporcionan evidencia de consistencia funcional entre el RTL y el netlist sintetizado para la secuencia de prueba utilizada.

Sin embargo, dicha comparación no constituye una equivalencia formal completa ni una verificación exhaustiva de todas las funcionalidades del juego.

La simulación gate-level se realizó sin anotación de retardos SDF, por lo que el cum-

plimiento de las restricciones temporales se evaluó mediante los reportes STA.

9.6. Trabajo futuro

A partir de los resultados obtenidos, se identifican distintas oportunidades para continuar el desarrollo del sistema.

Una primera línea de trabajo corresponde a optimizar las estructuras de almacenamiento del procesador, especialmente aquellas implementadas mediante flip-flops. El uso de memorias dedicadas o macros compatibles con la tecnología podría permitir reducir el área ocupada y la cantidad de celdas secuenciales, siempre que su integración resulte viable.

Otra posibilidad consiste en profundizar el análisis de los caminos críticos del procesador, evaluando modificaciones en la lógica de decodificación, las operaciones de la unidad aritmético-lógica y las rutas de acceso a memoria.

Estas optimizaciones podrían contribuir a mejorar el margen temporal y permitir períodos de reloj inferiores a los verificados durante el proyecto.

Desde el punto de vista funcional, sería conveniente ampliar el testbench para comprobar diferentes rondas del juego, la selección de objetivos, las respuestas correctas e incorrectas y el funcionamiento de los visualizadores de siete segmentos.

Asimismo, se podría implementar una verificación formal de equivalencia entre la descripción RTL y el netlist sintetizado.

Finalmente, sería relevante realizar una simulación gate-level con retardos SDF anotados, utilizando una herramienta compatible con los modelos temporales de GF180MCU, para complementar los resultados obtenidos mediante STA.

9.7. Conclusión final

En términos generales, el proyecto permitió completar un flujo de diseño ASIC para el sistema `Pochoco SoC`, desde la adaptación del RTL originalmente utilizado en FPGA hasta la obtención de una implementación física final.

El circuito implementado cumplió las restricciones temporales correspondientes al período objetivo de 100 ns y obtuvo resultados satisfactorios en las verificaciones físicas realizadas.

Los experimentos de optimización permitieron identificar compromisos entre área y temporización, mientras que las simulaciones RTL y gate-level proporcionaron evidencia de consistencia funcional para las condiciones ensayadas.

De esta manera, se logró una implementación ASIC completa desde el punto de vista del flujo digital desarrollado, junto con una evaluación cuantitativa de sus principales características físicas, temporales y funcionales.

10. Referencias bibliográficas

Referencias

[1] RISC-V International. *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture*. Especificación de la arquitectura RISC-V, incluyendo las instrucciones RV32I y RV32E. 2026. Disponible en: https://docs.riscv.org/

[2] LibreLane Project. *LibreLane Documentation*. Documentación oficial del flujo de diseño digital ASIC, desde RTL hasta implementación física. Disponible en: https://librelane.readthedocs.io/en/latest/

[3] GlobalFoundries y Google. *GlobalFoundries GF180MCU Open Source PDK*. Documentación de la tecnología de fabricación GF180MCU de 180 nm y sus bibliotecas. Disponible en: https://gf180mcu-pdk.readthedocs.io/

[4] GlobalFoundries y Google. *GF180MCU Process Design Kit*. Repositorio oficial del PDK utilizado para la implementación ASIC. Disponible en: https://github.com/google/gf180mcu-pdk

[5] Wolf, C. *Yosys Open SYnthesis Suite*. Herramienta de síntesis lógica de diseños digitales descritos en Verilog. Disponible en: https://yosyshq.net/yosys/

[6] The OpenROAD Project. *OpenROAD Documentation*. Documentación de las herramientas de implementación física, colocación de celdas, enrutamiento y análisis de temporización. Disponible en: https://openroad.readthedocs.io/en/latest/

[7] Ousterhout, J., y colaboradores. *Magic VLSI Layout Tool*. Herramienta de diseño de layout y verificación de reglas físicas (DRC). Disponible en: http://opencircuitdesign.com/magic/

[8] Graham, T. *Netgen: Network Comparison Tool*. Herramienta de comparación de redes eléctricas y verificación LVS. Disponible en: https://opencircuitdesign.com/netgen/

[9] Williams, S., y colaboradores. *Icarus Verilog*. Herramienta de compilación y simulación de diseños digitales en Verilog. Disponible en: https://steveicarus.github.io/iverilog/

[10] GTKWave Project. *GTKWave Documentation*. Herramienta de visualización y análisis de formas de onda digitales mediante archivos VCD. Disponible en: https://gtkwave.github.io/gtkwave/

[11] Equipo de desarrollo. *Proyecto 3: Arquitectura de Computadores. Implementación ASIC de Pochoco SoC*. Universidad de los Andes, Chile, 2026. Repositorio del proyecto: https://github.com/ChrisG925/Proyecto3_Arquitectura_computadores